

Programming with Data mid-term project

1. Introduction

In my project, I decided to analyze the relationship between a chosen country and its:

- Surface (SURF)
- Population (POP)
- Population density (DENS)
- Gross Domestic Product per Capita (GDP_PCAP)
- Gross Domestic Product (GDP)

in relation to the number of passengers (PSGR) carried using air transport. I chose to analyze the data for the year 2018 as it was the last year before the COVID pandemic began affecting air travel worldwide.

2. Project Background and Aims

The aviation field is of special interest to me as it is related to my current job. My initial consideration was to analyze the world's airport database to find some interesting relationships. I was especially interested in knowing which European airports are used by airlines. Additionally, one of the exercises during lectures required us to find an interesting dataset that could be used for Exploratory Data Analysis. One useful dataset related to aviation that I encountered provided statistics for some US airports regarding the number of passengers traveling from each airport. I considered analyzing city pairs, specifically the departure and destination airports, in terms of the number of passengers traveling. When considering city pairs, I was contemplating what factors make a specific route popular. I questioned whether it was influenced by: the population size in the area, the air travel distance between the cities or the wealth of the passengers. During my search for data, it proved difficult to find freely available and reliable datasets that could be used for my project. Although there were datasets available for purchase, their prices ranged from 300USD to 3000USD.

However, when I shifted the scale from individual airports to entire countries, I found that the World Bank website provided data on passenger traffic by air. The website offered detailed statistics for each country spanning many years in the past. From my point of view, the following statistics were particularly useful:

- The country's population.
- The country's surface area.
- The Gross Domestic Product (GDP) of the country and per capita.
- Most importantly, the number of passengers traveling by air.

For each statistic, I used the following dataframe names respectively:

- POP_2018: Data on population for each country.
- SURF_2018: Data on the country's surface area.
- DENS_2018: Data on population density for each country.
- GDP_2018: Data on Gross Domestic Product for each country.
- GDP_PCAP_2018: Data on Gross Domestic Product per capita for each country.
- PSGR_2018: Data on the number of passengers traveling by air for each country.

My analysis focused on comparing the population, surface area, population density, GDP per capita, GDP, and number of passengers traveling by air for each country.

Content of the report:

This document contains the following parts:

1. Data Acquisition
2. Data Cleansing, Processing, Plotting, and Analysis for each statistics pair, such as:
 - SURF to PSGR - Countries' surface area to air passengers
 - POP to PSGR - Countries' population to air passengers
 - DENS to PSGR - Countries' population density to air passengers
 - GDP_PCAP to PSGR - Countries' GDP per capita to air passengers
 - GDP to PSGR - Countries' GDP to air passengers
3. Ideas for Further Analysis with Examples
4. Conclusions
5. Information about the used data from an ethics perspective

Data acquisition

Where is my project's data coming from?

All data used in this project was publicly available from the following websites:

WORLD BANK (<https://www.worldbank.org/en/home>)

OUR AIRPORTS (<https://davidmegginson.github.io/ourairports-data>)

OUR WORLD IN DATA (<https://ourworldindata.org/>)

I believe that the information from the 'About' section on each website will describe the respective websites in the best way. Additionally, I have included the quoted Terms of Use for each website.

WORLD BANK (<https://www.worldbank.org/en/home>)

- **About World Bank** (<https://www.worldbank.org/en/who-we-are>)

With 189 member countries, staff from more than 170 countries, and offices in over 130 locations, the World Bank Group is a unique global partnership: five institutions working for sustainable solutions that reduce poverty and build shared prosperity in developing countries.

- **About World Bank Data** (<https://data.worldbank.org/>)

This site is designed to make World Bank data easy to find, download, and use. All of the data found here can be used free of charge with minimal restrictions.

- **Terms of use** (<https://data.worldbank.org/summary-terms-of-use>)

Unless indicated otherwise in the data or indicator metadata, you are free to copy, distribute, adapt, display or include the data in other products for commercial or noncommercial purposes at no cost under a Creative Commons Attribution 4.0 International License, with the additional terms(...)

OUR AIRPORTS (<https://davidmegginson.github.io/ourairports-data>)

- **About**([link](#))

OurAirports is a free site where visitors can explore the world's airports(...)

(...)Behind the fun and features, OurAirports exists primarily as a public good. When Australia forced the US government to shut down public access to its Digital Aeronautical Flight Information File (DAFIF) service in 2006, there was no longer a good source of global aviation data.

OurAirports started in 2007 primarily to fill that gap: we encourage members to create and maintain data records for airports around the world, and they manage over 40,000 of them. Many web sites, smartphone apps, and other services rely on OurAirport's data, which is all in the Public Domain (no permission required)."

- **Terms of use:**

"All data is released to the Public Domain, and comes with no guarantee of accuracy or fitness for use".

OUR WORLD IN DATA (<https://ourworldindata.org/>)

- **Website motto:** *Research and data to make progress against the world's largest problems.*

- **About** (<https://ourworldindata.org/about>)

Poverty, disease, hunger, climate change, war, existential risks, and inequality: The world faces many great and terrifying problems. It is these large problems that our work at Our World in Data focuses on.(..)We believe that a key reason why we fail to achieve the progress (..) is that (..) the important knowledge is often stored in inaccessible databases, locked away behind paywalls and buried under jargon in academic papers. (..)The goal of our work is to make the knowledge on the big problems accessible and understandable.

- **Terms of use** (<https://ourworldindata.org/faqs#can-i-use-or-reproduce-your-data>) *All data produced by Our World in Data is completely open access under the Creative Commons BY license. You have the permission to use, reproduce, and distribute it in any medium, provided the source and authors are credited. The data produced by third parties and made available by Our World in Data is subject to the license terms from the original third-party authors. We will always indicate the original source of the data in our documentation, so you should always check the license of any such third-party data before use and redistribution.*

The code

Importing modules

```
In [1]: import requests
import pandas as pd
import numpy as np
import shutil
import os
import re
import inspect

import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
```

Functions

The code below contains several functions that are used multiple times in the project. Furthermore, there are additional specific functions that can be found later in the code.

Helper functions

```
In [2]: def add_func_name():
        """Prints the current function being used when called - used mostly for debuggig."""
        if True:
            return '\n\t\t(' + inspect.currentframe().f_back.f_code.co_name + ' function.)'
```

```
In [3]: def print_red(text):
        """Makes the output string red - used to indicate an errors"""
        red_text = f"\033[91m{text}\033[0m"
        print(red_text)
```

```
In [4]: def max_col_OFF():
        "Switches off the restriction on the maximum number of columns when renders the dataframe"
        pd.set_option('display.max_columns', None) # Removing restriction on the number of columns to display

        def max_col_ON():
            "Switches back to on the maximum number of columns when dataframe is rendered in the output"
            pd.reset_option('display.max_columns')
```

```
In [5]: def max_rows_OFF():
        "Limit on the number of rows rendered into the output is lifted"
        pd.set_option('display.max_rows', None)

        def max_rows_ON():
            "Limit on the row number is on again"
            pd.reset_option("display.max_rows")
```

```
In [6]: def print_download_error_msg(file_path,e=""):
        print(print_red(f"ERROR - download failed. Using backup file {file_path}.\n\n{str(e)}"))
```

```
In [7]: def print_file_downloaded_msg(file_path):
        print(f'{file_path} downloaded!')
```

Functions - data cleansing

```
In [8]: def remove_not_country_rows(df):
        """DATA CLEANSING - Remove rows that do not represent countries."""
        return df[df['Country Code'].isin(countries['alpha-3'])]
```

Functions - plotting

```
In [9]: def convert_wide_to_long_format_WORDL_BANK_dat(df, y_label="y_lable_missing",start_year=1960, end_year=2023):
        """ PLOTTING - converts form wide to long format the datagrame downloaded from the World Bank data website"""
        df_copy = df
        df_melted = df_copy.melt(
            id_vars=['Country Name', 'Country Code', 'Indicator Name', 'Indicator Code'],
            value_vars=[str(i) for i in range(start_year, end_year)],
            var_name='Year',
            value_name=y_label # This is now a unique name
        )

        # Converting 'Year' to integer to enable the correct plot
        df_melted['Year'] = df_melted['Year'].astype(int)

        return df_melted
```

```
In [10]: def plot_worldbank_long_format(df, to_display_list, y_label ="y_lable_missing"):
        df_selected_countries = df[df['Country Name'].isin(to_display_list )]
```

```

### Plotting PSGR(air passenger numbers) vs year.
g=sns.relplot(kind="line",x="Year",y=y_label, hue="Country Name",errorbar=None, data=df_selected_countries )
ax = g.axes[0, 0]

# Reformatting y axis to have have comma every 3 significant digits
ax.yaxis.set_major_formatter(mticker.StrMethodFormatter('{x:,.0f}'))
plt.show()

```

Downloading data

When this notebook is run, initially, the program will attempt to download the data using the provided URL. If that process fails, it will fallback to using the "Backup_data" folder and utilize the backup files. I have tested this functionality in situations where there is no internet access or the URL is invalid. In case any issues arise, disabling the internet access might help resolve the problem. I believe that my code is robust enough to ensure the notebook functions properly regardless of the circumstances.

Countries list

In my project, I also utilize static databases that do not require downloading from the internet. These databases are unlikely to change within the context of my project. The file 'countries.csv' contains information about every currently existing country.

For my specific needs, I only require the official country name and its three-letter alphanumeric code from this source.

```

In [11]: countries_raw = pd.read_csv("Static_data/countries.csv")
countries = pd.DataFrame(countries_raw, columns=["name", "alpha-3"])

```

```

In [12]: countries_raw.head(3)

```

```

Out[12]:

```

	name	alpha-2	alpha-3	country-code	iso_3166-2	region	sub-region	intermediate-region	region-code	sub-region-code	intermediate-region-code
0	Afghanistan	AF	AFG	4	ISO 3166-2:AF	Asia	Southern Asia	NaN	142.0	34.0	NaN
1	Åland Islands	AX	ALA	248	ISO 3166-2:AX	Europe	Northern Europe	NaN	150.0	154.0	NaN
2	Albania	AL	ALB	8	ISO 3166-2:AL	Europe	Southern Europe	NaN	150.0	39.0	NaN

Downloading data - airports data

All data downloaded from the internet and data extracted from the backup ZIP files will be stored in "Data\" folder

```
In [13]: # Creates data folder if do not exists
os.makedirs("Data", exist_ok=True)
```

Below is the function responsible for downloading the file from the internet or loading backup file and feed the data into the Pandas dataframe.

```
In [14]: def download_csv_and_return_dataframe(URL, filename):
    try: # Tries to download the file - if successful the file will be saved in the "Data/" folder
        response = requests.get(URL)
        folder = "Data/"

        with open(folder+filename, 'wb') as f:
            f.write(response.content)

        print_file_downloaded_msg(folder+filename)

    except Exception as e:
        # The connection with URL was not possible - source folder changed to the backup one
        folder = "Backup_data/"
        print_download_error_msg(folder+filename,e)

    try: # The source "folder" is used to upload the file into the dataframe
        return pd.read_csv(folder+filename)

    except Exception as e:
        # The file read by Pandas is corrupted - it happens when the URL turns out to be incorrect
        # Reverts to backup folder for the data.
        print_red(f'{str(e)}\nUsing backup data: {folder+filename}!')
        folder = "Backup_data/"

    return pd.read_csv(folder+filename)
```

```
In [15]: # Attempts to retrieve the response from the specified URL
# Establishing the name for the file contained in the response
URL = "https://ourairports.com/airports.csv"
filename = "airports_databases_raw.csv"
```



```
airports = download_CSV_and_return_dataframe(URL,filename)
airports.shape
```

Data/airports_databas_raw.csv downloaded!
(35144, 23)

Out[15]:

The airports database contains tens of thousands of data points, as there are this many airports of different sizes on the planet.

Downloading data - runways data

The runways dataframe is acquired using the same function as the airports dataframe,

For clarification, the runways data was intended to be used for extensive analysis. However, I dropped the idea since the project would become too complex. I only used the runway briefly at the end of the project to show the potential for further data exploration.

```
In [16]: # Getting runway data
URL = 'https://davidmegginson.github.io/ourairports-data/runways.csv'
filename = 'runways_raw.csv'
runways =download_CSV_and_return_dataframe(URL, filename)
runways.shape
```

Data/runways_raw.csv downloaded!
(44940, 20)

Out[16]:

```
In [17]: max_col_ON() # Lifts limit on the number of columns rendered
runways.head(3)
```

Out[17]:

	id	airport_ref	airport_ident	length_ft	width_ft	surface	lighted	closed	le_ident	le_latitude_deg	le_longitude_deg	le_elevation_ft	le_heading
0	269408	6523	00A	80.0	80.0	ASPH-G	1	0	H1	NaN	NaN	NaN	
1	255155	6524	00AK	2500.0	70.0	GRVL	0	0	N	NaN	NaN	NaN	
2	254165	6525	00AL	2300.0	200.0	TURF	0	0	01	NaN	NaN	NaN	

In [18]: max_col_OFF() # Restores the Limit back

Similarly to the airports, the runways dataframe also contains tens of thousands of data points. However, the runways data is smaller in size compared to the airports dataframe. This is because the airports dataframe encompasses all types of landing fields, including helipads and very small airfields, which can be temporarily unavailable or closed. On the other hand, the runways dataframe focuses specifically on well-maintained and larger airports.

WORLD BANK data

<https://data.worldbank.org/>

Below are the common parameters applicable to all data downloaded from the World Bank website. Params variable is used for each download.

```
In [19]: params={
          'downloadformat': 'csv'
        }
```

WORLD BANK - Passengers traffic per country (PSGR)

<https://data.worldbank.org/indicator/IS.AIR.PSGR>

In the case of downloading data from the World Bank, the process is similar to that of airports and runways data. However, there are a few key differences. Firstly, the downloaded file from the URL is in ZIP format, and it needs to be extracted. Additionally, the ZIP file contains three files, of which only one contains the actual data. The functions provided below handle the process of extracting the data from the ZIP file and transforming it into a dataframe.

```
In [20]: def return_data_filename__worldbank(search_string, data_folder):
          """The data downloaded from the World Bank website is in the form of a ZIP file.
          Once extracted, it consists of two metadata files and one CSV file containing the data.
          This function retrieves the name of the CSV file."""

          files = shutil.os.listdir(data_folder)
          for f_name in files:
              if re.search(search_string, f_name) and not re.match(r"Metadata", f_name):
                  return f_name
          print_red("ERROR: No match.")
          return None
```

```

In [21]: def download_WorldBank_ZIP_and_return_dataframe(URL, params, filename, key_string):
        """
        This function attempts to download the dataset files in the form of a ZIP file from the World Bank website.
        If the download fails, it falls back to using files stored in the 'Backup_data' folder. In both cases, the ZIP file will be e

        The 'key_string' parameter (str) is used to locate the dataset's CSV file once the ZIP file is extracted.
        Each ZIP file contains two metadata files and one file containing the data.

        The function" creates a file with the specified 'filename' and
        returns the Pandas dataframe.
        """

        try: # Tries to download and save the file
            response = requests.get(URL,params)
            folder= "Data/"
            with open(folder+filename, 'wb') as f:
                f.write(response.content)

            print_file_downloaded_msg(folder+filename)

        except Exception as e: # Download failed - reverting to the backup folder for data
            folder= "Backup_data/"
            print_download_error_msg(folder+filename,e)

        # Make Data/Extracted__ folder
        extracted_folder = "Data/Extracted__" +filename[:-4]
        os.makedirs(extracted_folder, exist_ok=True)

        # Unzip ZIP file into Data/Extracted__ folder
        if os.path.exists(folder + filename):
            try: ## Tries to unzip ZIP file
                shutil.unpack_archive(folder + filename, extracted_folder,'zip')

                print(f'"{filename}" extracted into {extracted_folder} ' + add_func_name())
            except Exception as e: # Extract failed - ZIP file is corrupted, maybe the URL is incorrect

                folder= "Backup_data/"
                print_red(f'{str(e)}\nReverting to backup ZIP file {folder + filename}!')
                shutil.unpack_archive(folder + filename, extracted_folder,'zip')

        #####

```

```

# Finding the CSV data file and returning its file name
CSV_filename = return_data_filename__worldbank(key_string, extracted_folder)

# Constructing the file path by joining the extract folder and the data filename

return pd.read_csv(f"{extracted_folder}/{CSV_filename}", skiprows=3)

```

```

In [22]: URL='https://api.worldbank.org/v2/en/indicator/IS.AIR.PSGR'
filename = "worldbank_PSGR.zip"
worldbank_PSGR_raw = download_WorldBank_ZIP_and_return_dataframe(URL,params,filename, "PSGR")
del URL, filename
worldbank_PSGR_raw.shape

```

Data/worldbank_PSGR.zip downloaded!
 "worldbank_PSGR.zip" extracted into Data/Extracted__worldbank_PSGR
 ("download_WorldBank_ZIP_and_return_dataframe" function.)

Out[22]: (266, 68)

```

In [23]: max_col_OFF()
worldbank_PSGR_raw.head(3)

```

	Country Name	Country Code	Indicator Name	Indicator Code	1960	1961	1962	1963	1964	1965	1966	1967	1968	1969	1970	1971	1972
0	Aruba	ABW	Air transport, passengers carried	IS.AIR.PSGR	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	Africa Eastern and Southern	AFE	Air transport, passengers carried	IS.AIR.PSGR	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3418300.0	3775300.0	4074500.0
2	Afghanistan	AFG	Air transport, passengers carried	IS.AIR.PSGR	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	84700.0	97400.0	104900.0

```
In [24]: max_col_ON()
```

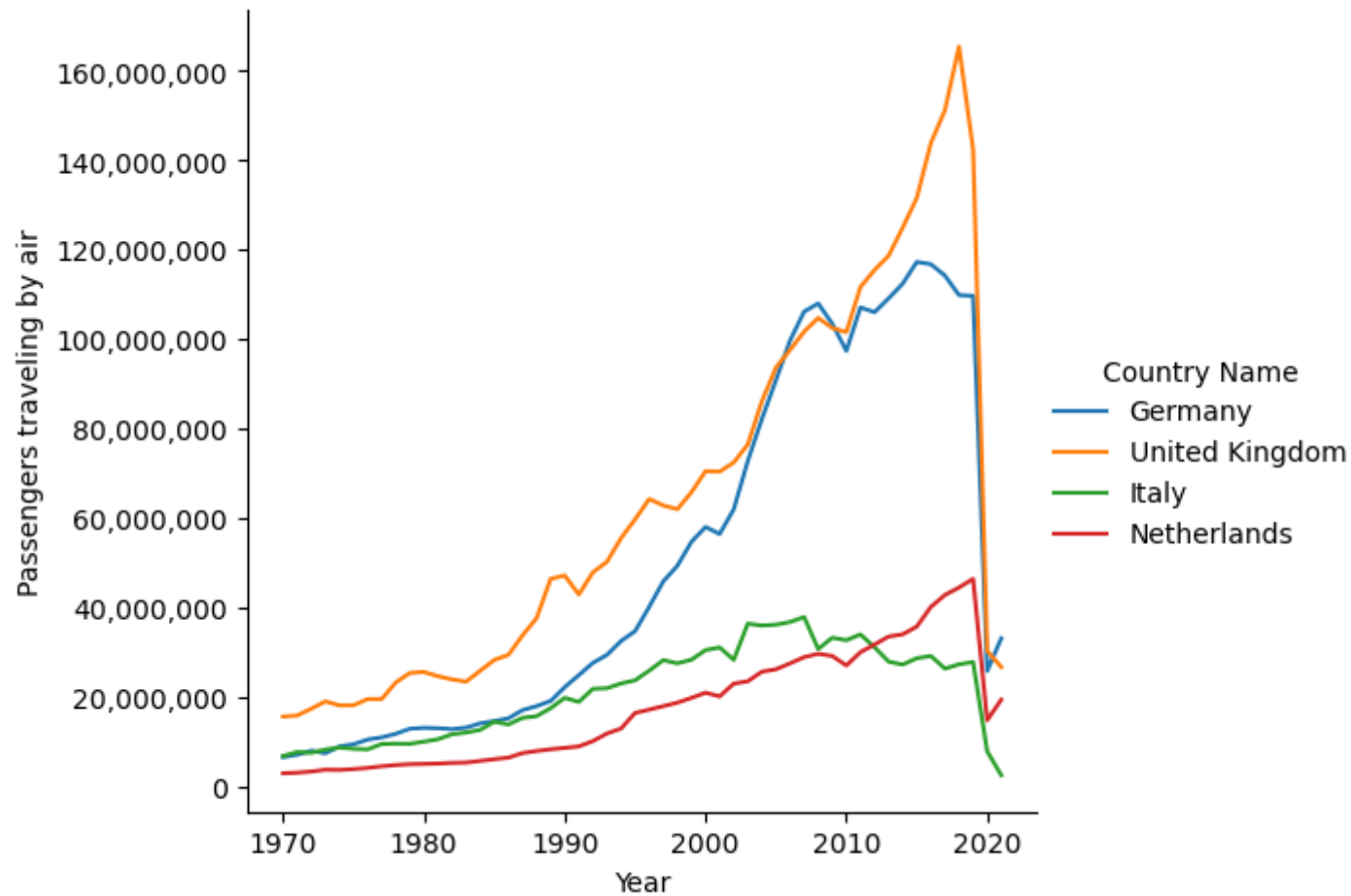
Here, the dataframe contains only a few hundred datapoints. Ideally, it should have as many rows as there are countries for the period starting from the year 1960. However, this data also includes information grouped into regions and other types of categories.

Furthermore, the data is presented in the 'wide format,' where the years are represented in columns and countries in rows.

I have created the function that converts 'wide format' to long format to be usable with World Bank data. When used it enables the plot World Bank dataframes

```
In [25]: countries_to_display = ['Italy',"Germany", "Netherlands","United Kingdom"]

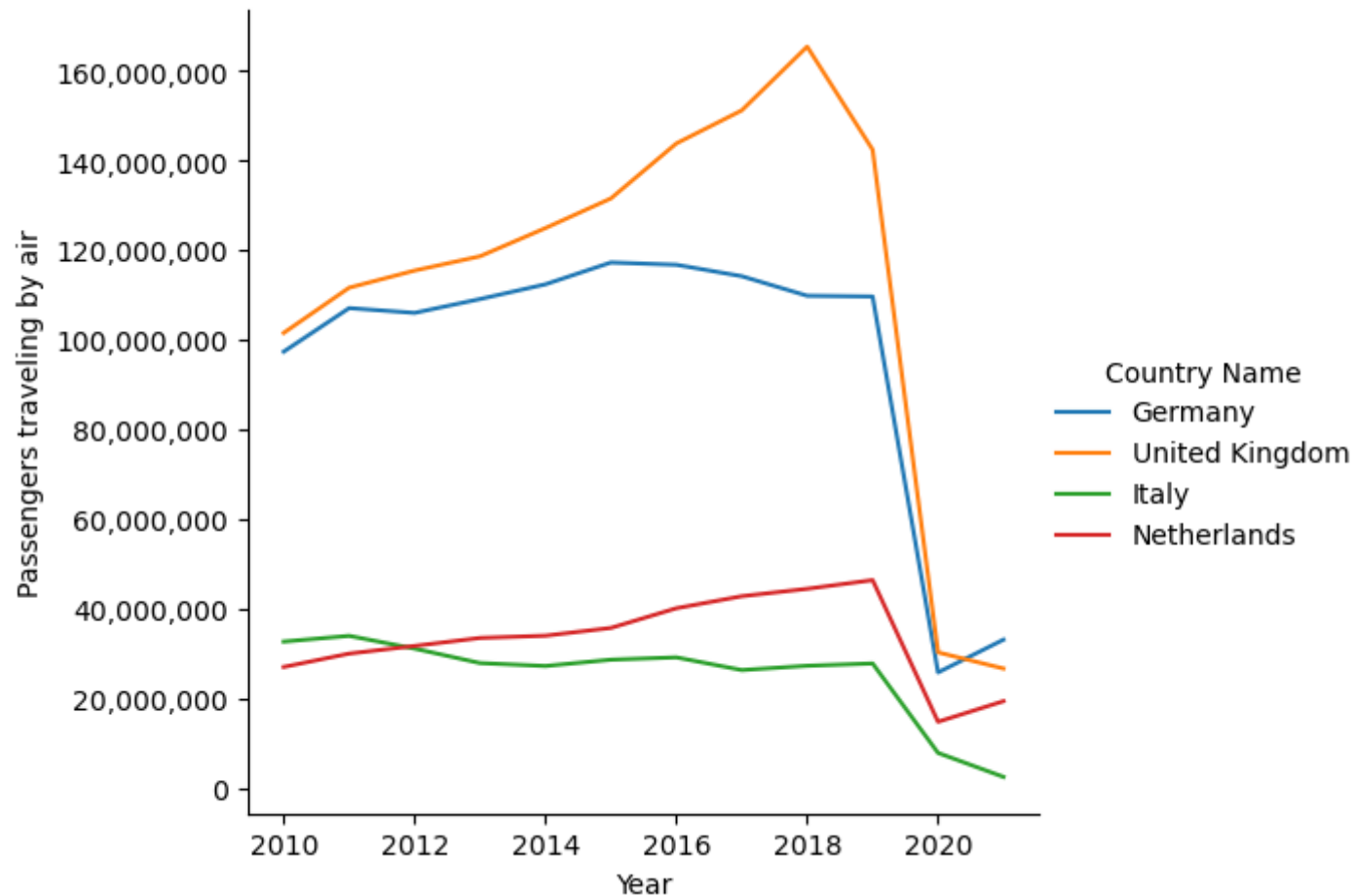
# Conversion of 'wide' to 'Long' format
y_label = "Passengers traveling by air"
worldbank_PSGR_long = convert_wide_to_long_forma_WORLD_BANK_dat(worldbank_PSGR_raw,y_label=y_label)
plot_worldbank_long_format(worldbank_PSGR_long, countries_to_display,y_label=y_label)
```



The plot above displays the number of passengers carried in a given year for a specific country. It is worth to notice the significant drop in air passenger traffic that occurred in the year 2020.

In the section below, the same relationship is presented for the same countries, however, the data is limited to the years 2010-2023.

```
In [26]: worldbank_PSGR_long_2010to2023 = convert_wide_to_long_forma_WORDL_BANK_dat(worldbank_PSGR_raw, start_year=2010, end_year=2023, y_
plot_worldbank_long_format(worldbank_PSGR_long_2010to2023, countries_to_display, y_label=y_label)
del y_label
```



The sharpest drop was between the years 2019 and 2020, which corresponds precisely to the start of the COVID pandemic, officially recognized on March 11, 2020. Around that time, many countries severely restricted air travel, reducing it to almost nothing.

I chose the year 2018 as my reference to provide some margin.

WORLD BANK - population total

<https://data.worldbank.org/indicator/SP.POP.TOTL>

In this case, the file containing population data is being downloaded. The process utilizes the same function as the previous dataframe retrieval function.

```
In [27]: URL = "https://api.worldbank.org/v2/en/indicator/SP.POP.TOTL"
filename = "worldbank_POP_TOTL.zip"
worldbank__POP_raw = download_WorldBank_ZIP_and_return_dataframe(URL,params,filename, "POP")
del URL, filename
worldbank__POP_raw.shape
```

Data/worldbank_POP_TOTL.zip downloaded!

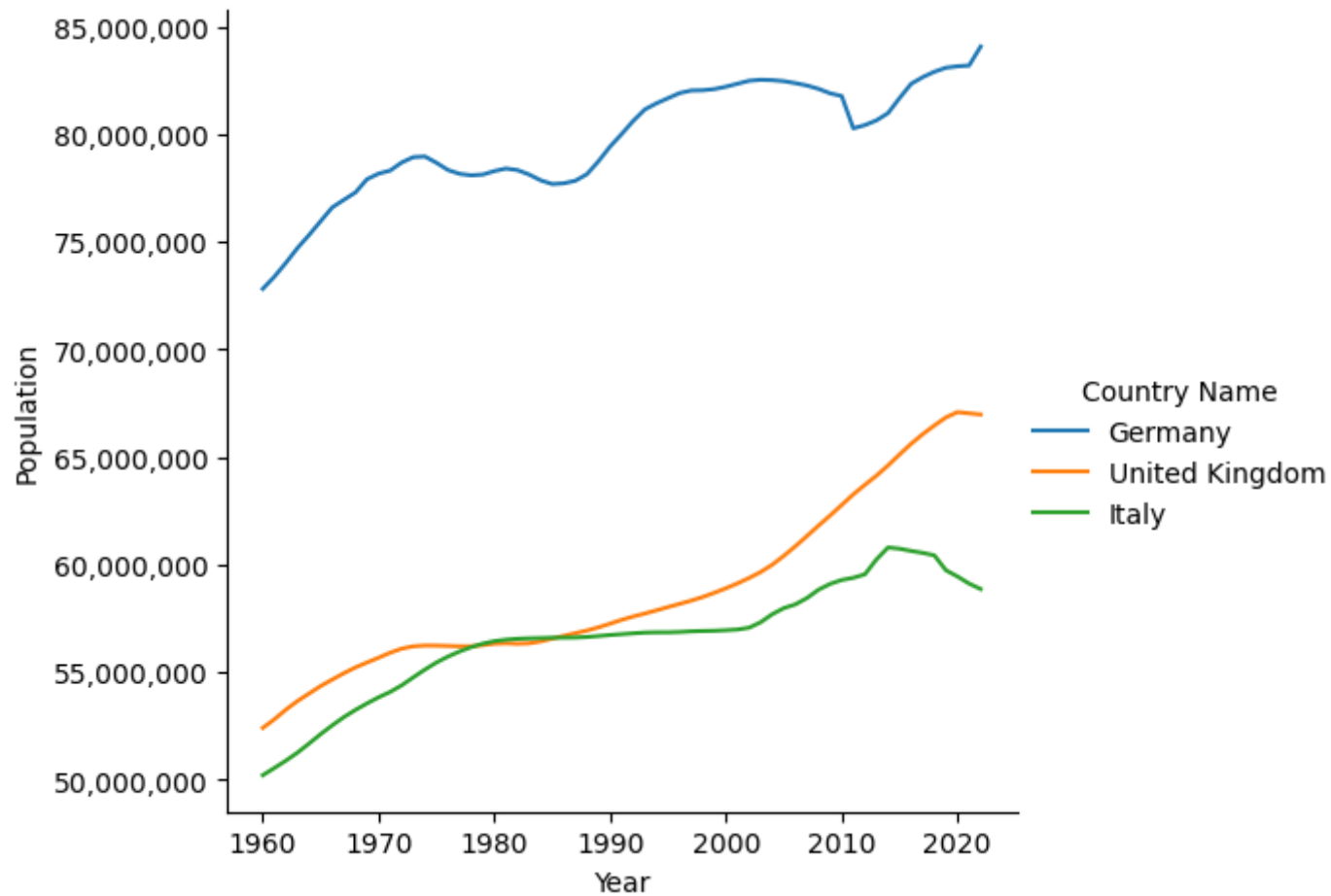
"worldbank_POP_TOTL.zip" extracted into Data/Extracted__worldbank_POP_TOTL
("download_WorldBank_ZIP_and_return_dataframe" function.)

Out[27]: (266, 68)

Above, the shape of the data is the same as in the previous dataframe.

Below, the plot illustrates the population change for selected countries since the year 1960. The graph provides an overview of how the population has evolved over time for the chosen countries.

```
In [28]: countries_to_display = ['Italy','Germany','United Kingdom']
y_label = "Population"
w_long_2 = convert_wide_to_long_format_WORLD_BANK_dat(worldbank__POP_raw, y_label=y_label)
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)
del y_label
```

WORLD BANK - Countries surface areas

<https://data.worldbank.org/indicator/AG.SRF.TOTL.K2>

Below is the code responsible for acquiring the data containing the surface area of each country. The process remains the same as before, utilizing the previously established methods and functions.

```
In [29]: URL = "https://api.worldbank.org/v2/en/indicator/AG.SRF.TOTL.K2"
filename = "worldbank_SRF_TOTL.zip"
worldbank__SURF_raw = download_WorldBank_ZIP_and_return_dataframe(URL,params,filename, "SRF")
del URL, filename
```

```
worldbank__SURF_raw.shape
```

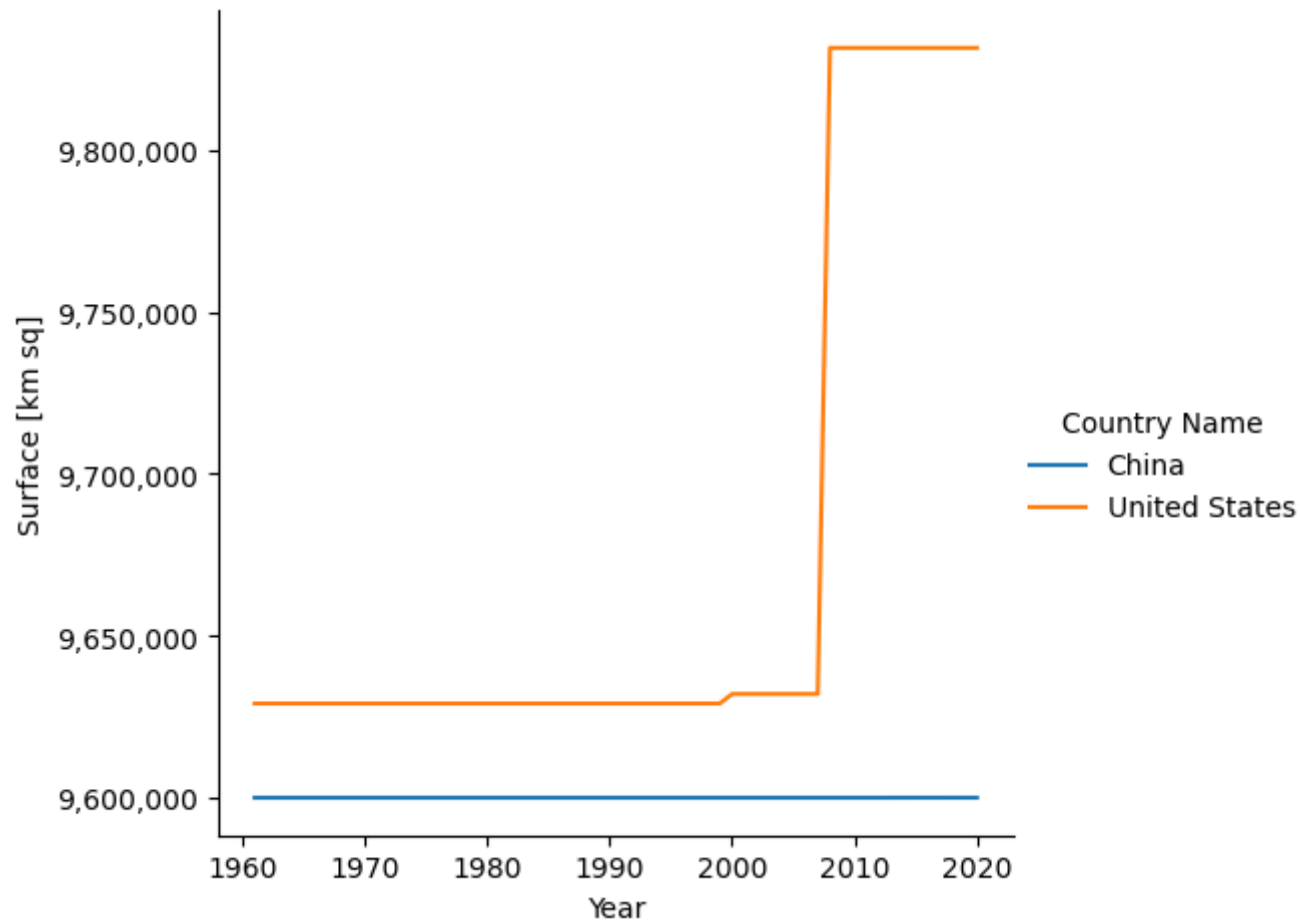
Data/worldbank_SRF_TOTL.zip downloaded!

"worldbank_SRF_TOTL.zip" extracted into Data/Extracted__worldbank_SRF_TOTL
("download_WorldBank_ZIP_and_return_dataframe" function.)

Out[29]: (266, 68)

Once again the the shape of the data is the same as in previous dataframe.

```
In [30]: countries_to_display = ['United States', "China"]
y_label = "Surface [km sq]"
w_long_2 = convert_wide_to_long_format_WORLD_BANK_dat(worldbank__SURF_raw, y_label=y_label)
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)
del y_label
```



As expected, China and the United States have similar surface areas. Interestingly, there is a shift in the surface area of the United States in this dataset. I was not able to find the exact reason for this change in the data. My theory was that some state was included in the US territory, which was not previously counted as part of it. However, the last state that joined US territory was Hawaii in the year 1959.

WORLD BANK - GDP per capita (GDP_PCAP)

<https://data.worldbank.org/indicator/NY.GDP.PCAP.CD>

```
In [31]: URL = "https://api.worldbank.org/v2/en/indicator/NY.GDP.PCAP.KD"
filename = "worldbank_GDP_PCAP.zip"
worldbank__GDP_PCAP_raw = download_WorldBank_ZIP_and_return_dataframe(URL,params,filename, "PCAP")
```

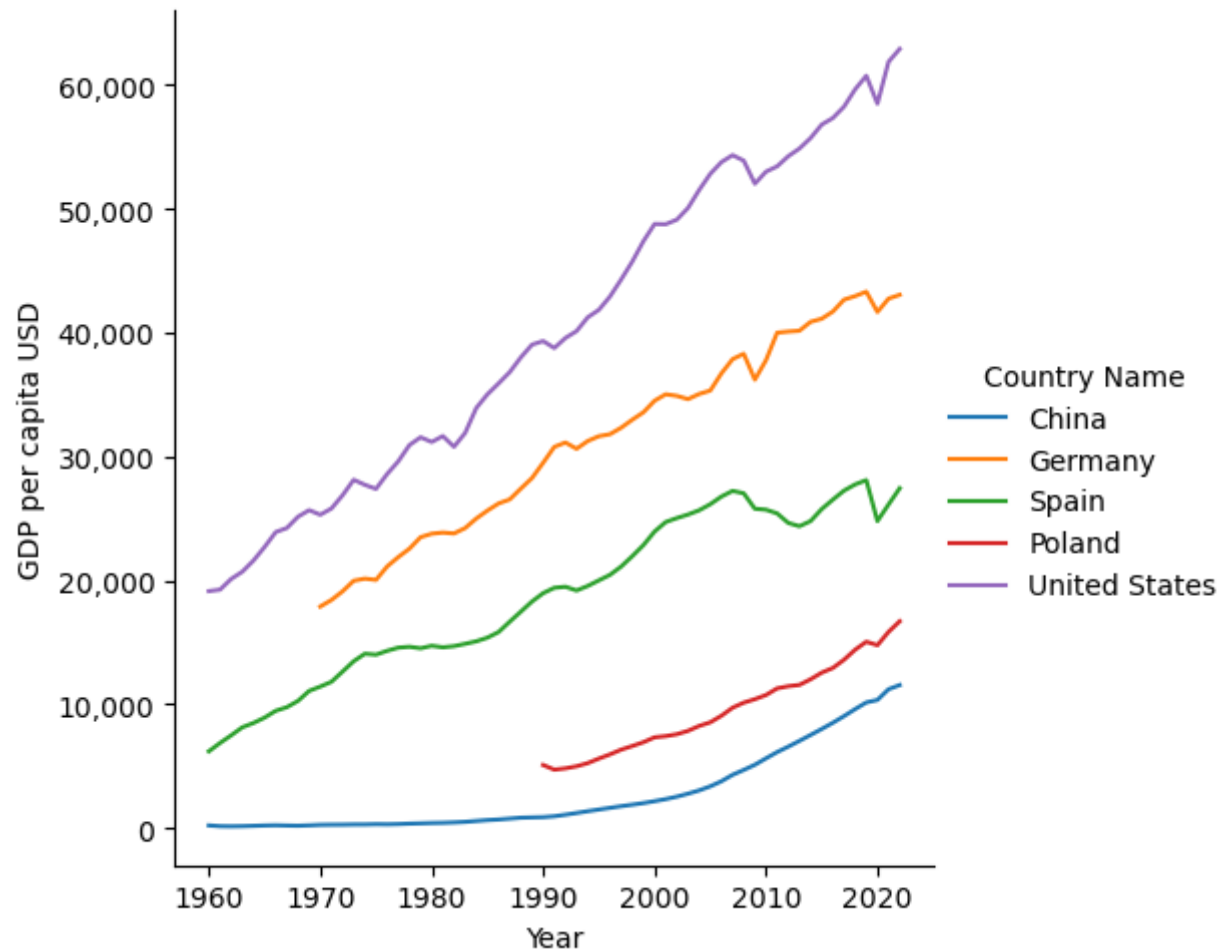
```
del URL, filename
worldbank__GDP_PCAP_raw.shape
```

Data/worldbank_GDP_PCAP.zip downloaded!

"worldbank_GDP_PCAP.zip" extracted into Data/Extracted__worldbank_GDP_PCAP
("download_WorldBank_ZIP_and_return_dataframe" function.)

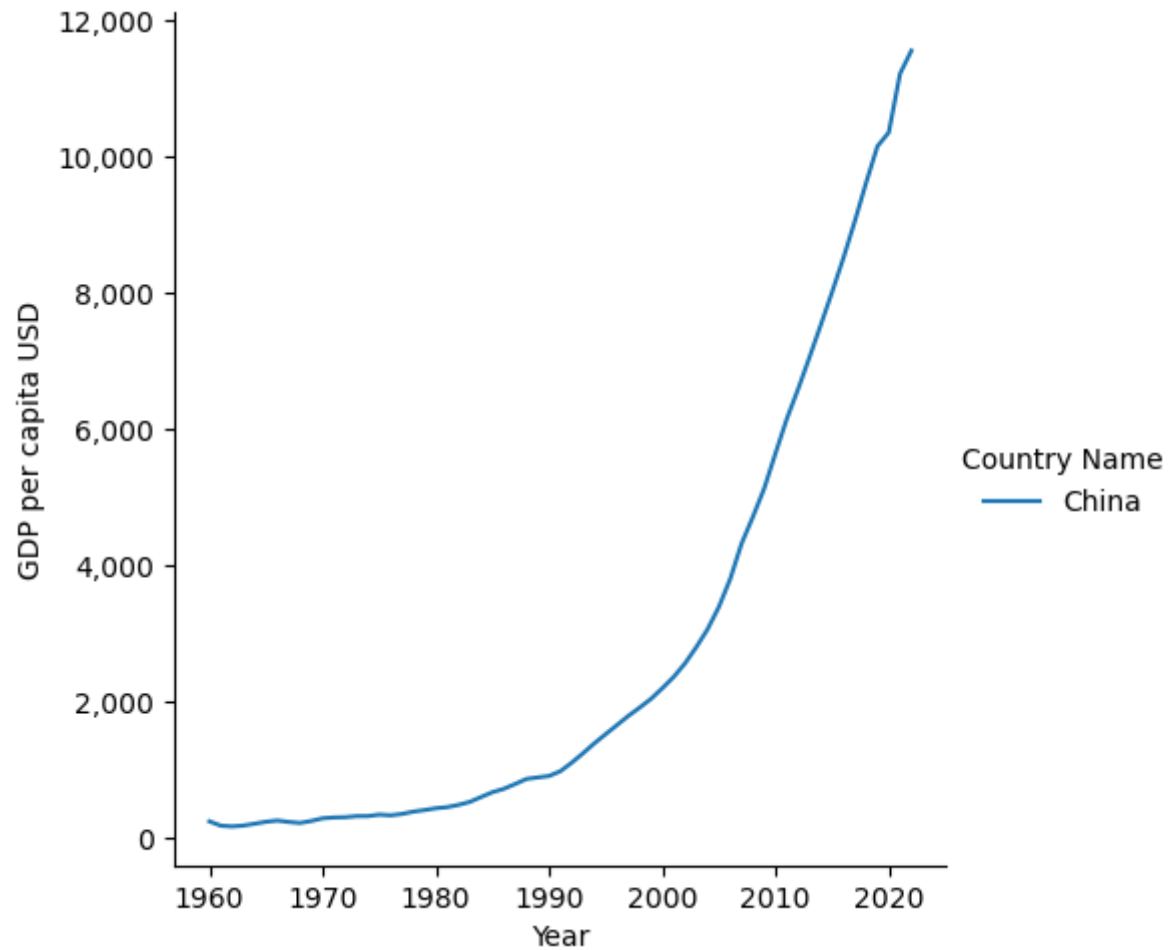
Out[31]: (266, 68)

```
In [32]: countries_to_display = ['United States', "China", "Germany", "Poland", "Spain"]
y_label = "GDP per capita USD"
w_long_2 = convert_wide_to_long_format_WORLD_BANK_dat(worldbank__GDP_PCAP_raw, y_label=y_label)
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)
del y_label
```



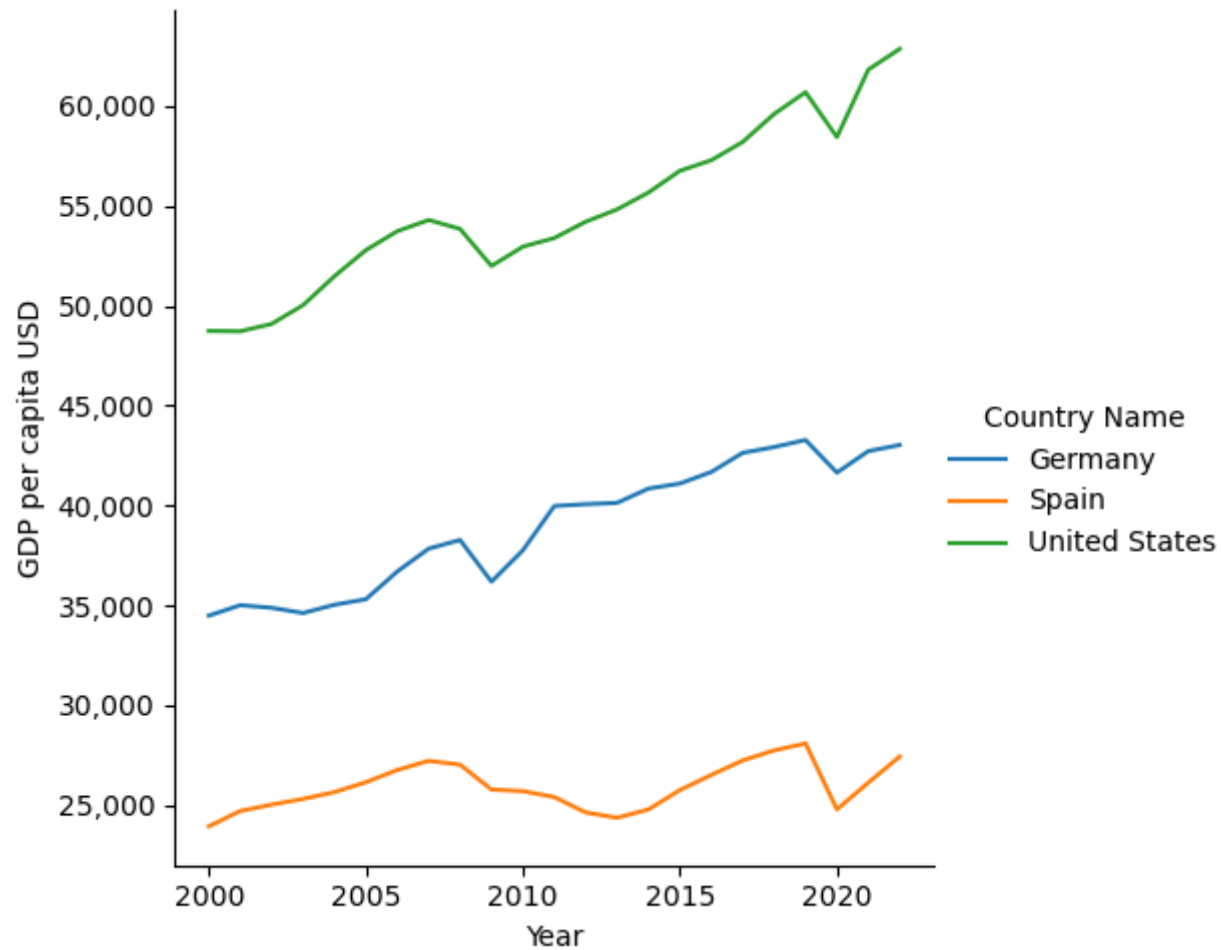
Looking on this graph It seems that the GDP per capita is steadily rising for some countries. There are two interesting features on this chart. The Chinas rise and also the year 2008 financial crisis. More detailed charts are below:

```
In [33]: countries_to_display = ["China"]
y_label = "GDP per capita USD"
w_long_2 = convert_wide_to_long_format_WORDL_BANK_dat(worldbank__GDP_PCAP_raw, y_label=y_label)
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)
del y_label, countries_to_display
```



The rise of China is almost exponential in presented period.

```
In [34]: countries_to_display = ['United States', "Germany", "Spain"]
y_label = "GDP per capita USD"
w_long_2 = convert_wide_to_long_format_WORLD_BANK_dat(worldbank_GDP_PCAP_raw, y_label=y_label, start_year=2000)
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)
del y_label, countries_to_display
```



Actually, there are two visible drops in the GDP per capita: one in 2008 and the second one in 2019 when COVID struck.

World Bank - Gross Domestic Product

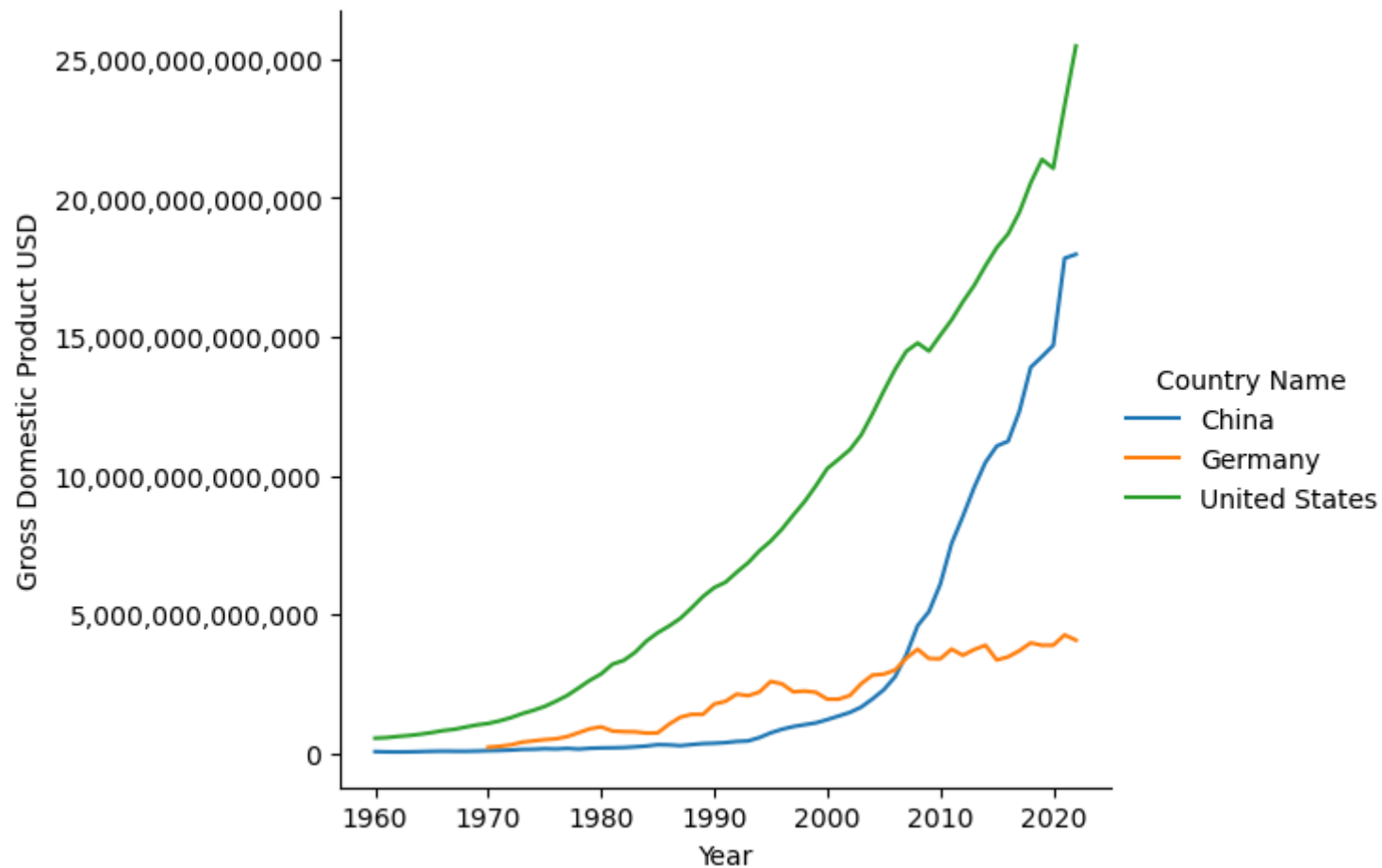
<https://data.worldbank.org/indicator/NY.GDP.MKTP.CD>

```
In [35]: URL = "https://api.worldbank.org/v2/en/indicator/NY.GDP.MKTP.CD"
filename = "worldbank_GDP.zip"
worldbank__GDP_raw = download_WorldBank_ZIP_and_return_dataframe(URL,params,filename, "MKTP")
del URL, filename
worldbank__GDP_raw.shape
```

```
Data/worldbank_GDP.zip downloaded!  
"worldbank_GDP.zip" extracted into Data/Extracted__worldbank_GDP  
  ("download_WorldBank_ZIP_and_return_dataframe" function.)
```

```
Out[35]: (266, 68)
```

```
In [36]: countries_to_display = ['United States', "China", "Germany"]  
y_label = "Gross Domestic Product USD"  
w_long_2 = convert_wide_to_long_forma_WORLD_BANK_dat(worldbank__GDP_raw, y_label=y_label)  
plot_worldbank_long_format(w_long_2, countries_to_display, y_label=y_label)  
del y_label
```



Similarly to GDP per capita the rise of the GDP is visible. The rise of China's GDP here is even more clearly visible.

Our World in Data

<https://ourworldindata.org/grapher/political-regime>

```
In [37]: legend = {  
    0:"Closed autocracy",  
    1:"Electoral autocracy",  
    2:"Electoral democracy",  
    3:"Liberal democracy",  
}  
regime = pd.read_csv("Static_data/political_regime.csv")  
regime.shape  
# print(regime)
```

Out[37]: (30766, 4)

I intended to use this dataset to compare the relationship between airport infrastructure and a country's political regime. My expectation was to observe a high number of airports in countries with a long history of liberal democracy, and a lack of airports, except the large civilian and military airports in autocracies. This distinction should be particularly noticeable in the General Aviation sector, which refers to the utilization of small or private airplanes. At the end of this report, I provide an example to illustrate this point. Although I haven't utilized this dataset in my project, it could be interesting to delve deeper into it for further exploration.

Data analysis

Each row of the World Bank CSV file contains data that are not always country-specific - they may represent regions or other aspects. The function used below removes those rows that contain non-country data.

```
In [38]: worldbank__GDP = remove_not_country_rows(worldbank__GDP_raw)  
worldbank__GDP_PCAP = remove_not_country_rows(worldbank__GDP_PCAP_raw)  
worldbank__SURF = remove_not_country_rows(worldbank__SURF_raw)  
worldbank__PSGR = remove_not_country_rows(worldbank__PSGR_raw)  
worldbank__POP = remove_not_country_rows(worldbank__POP_raw)
```

At this point, the following dataframes have been uploaded into memory:

- **airports** - contains world airport data.
- **runways** - contains more detailed information about the runways at each airport, including their * **length** and width.
- **worldbank_PSGR** - holds data on passenger traffic per country.
- **worldbank_SURF** - contains information about the surface area of countries.
- **worldbank_GDP_PCAP** - provides data on Gross Domestic Product (GDP) per capita for most countries.

Functions related to the data analysis

```
In [39]: def get_series_for_selected_year(df, year):
        """
        Extracts a column data in a form of a pd.Series from a World Bank data DataFrame for a given year.
        The index of the Series is set as the 'Country Name' column of the DataFrame.
        This function is designed to work specifically with certain World Bank CSV files.
        """
        year = str(year)

        series = pd.Series(df[year])
        series.index = df['Country Name']

        # Convert values to int and handle NaN values
        series = series.apply(lambda x: int(x) if pd.notna(x) else x)

        return series
```

```
In [40]: def print_world_population_factor(world_pop_in_year, pop_column, prnt=True):
        """
        Calculates the factor between the sum of the population for countries given a Series or DataFrame column and the total world
        Optionally, it can print the output in a formatted way. """
        factor = pop_column.sum()/world_pop_in_year

        if prnt:
            print('{:.0%}'.format(factor)+f' of world population in year {year}', add_func_name())

        return factor
```

The generate_align_factor function

The **generate_align_factor** function is the most important function in my project. It calculates the relationship between the data in colA and colB using an arbitrary reference country. For example, it calculates the ratio between the GDP of Canada and its Passenger traffic, and stores this ratio

for later use. The factor for the reference country is set to 1.

Next, the align_factor is calculated for selected parameters (e.g., GDP and PSGR) for each country in relation to the reference country (Canada). This allows for a comparison of the relationship between GDP and PSGR for each country.

The function is designed in such a way that when the result of GDP/PSGR multiplied by the reference_ratio is above 1, the factor is the result of the equation. However, if it is below 1, the division is inverted, meaning PSGR is divided by GDP instead of the other way around, and a negative sign is applied. This implies that the align_factor falls within the range of 1 to infinity and -1 to negative infinity.

```
In [41]: def generate_value(row, colA, colB, reference_ratio):
        """Internal function for generate_align_factor function below"""
        if row[colA]/(row[colB]*reference_ratio) > 1:
            align_factor = 1+(row[colA] / (row[colB] * -reference_ratio))
        else:
            align_factor = -1+(row[colB] / row[colA] * reference_ratio)
        return align_factor.round(2)

def generate_align_factor(df,colA,colB,ref_country=None,outputColName=None,): #generate_align_factor
    """
    df - that need to contain col A and B and index that is 'Country Name'
    colA,colB - column names to calculate the factor (e.g. GDP_2018, SURF_2018)
    ref - reference "Country Name" (e.g Canada)

    returns: df,reference_ratio, numA, numB
    numA,numB - those are values for colA and colB of the reference country
    """

    # Compare
    if outputColName:
        newName=outputColName
    else:
        partA = re.sub("_\d{4}", "", colA)
        newName = '_' .join([partA,'to',colB])

    # Calculate reference's country a reference ratio (GDP/passengers)
    # numA and numB are the values for reference country (i.e. Canada) and the respective columns(i.e. GDP and PSGR)
    if ref_country:

        numA = df.loc[ref_country, colA]
        numB = df.loc[ref_country, colB]
        reference_ratio = numA / numB
```

```

else:
    numA = None
    numB = None
    reference_ratio=1

    # Calculate ratio for each country and divide by reference ratio
    df[newName] = df.apply(generate_value, args=(colA, colB, reference_ratio), axis=1)

    return df,reference_ratio, numA, numB

```

```

In [42]: ## Stores dropped rows for which factor is exceeded
def exclude_to_high_factor_countries(df,factor_col_name, factor):
    """
    This function filters out countries that fall outside the align factor
    (check generate_align_factor function - above)

    Returns the:
        - the original dataframe,
        - the adjusted dataframe - the data frame which do not contain any more countries below the threshold
        - the dataframe of dropped rows
    """
    countries_to_exclude = df[(df[factor_col_name]< -factor) | (df[factor_col_name]> factor)
                             ].index.tolist()

    df_copy = df.copy() # To avoid modification of the original data

    # Drop the rows
    df_adjusted = df_copy.drop(countries_to_exclude)

    # Creating a dataframe of dropped rows
    dropped_countries_df = df_copy.loc[countries_to_exclude]

    return df, df_adjusted, dropped_countries_df

```

```

In [43]: def df_row_float_to_int(df,index_name):
    """Converts type of the data in a row from float to int"""
    # Dataframe has to be transposed to change the type of the row.
    df_t = df.transpose()
    df_t[index_name] = df_t[index_name].astype(int)
    df = df_t.transpose()

    return df

```

```
In [44]: def df_row__float_to_percent(df, index_name, decimal_pts=0): #df_row__float_to_percent
        """
        Converts the values in a specific row of dataframe to %.

        df(Dataframe) -the input dataframe.
        index_name (str): The name of the row to convert to percentages.
        decimal_pts (int, optional): The default is 0 . Number of decimal points to round the percentages to.,

        Returns converted dataframe
        """
        df_t = df.transpose()

        # Define a conversion function that handles different cases based on value
        def convert_to_percentage(x):
            if abs(x) > 0.01 :
                return str(int(x * 100)) + '%'
            else:
                return str(round(x * 100, 1)) + '%'

        # Convert the specified row values to percentages
        df_t[index_name] = df_t[index_name].apply(convert_to_percentage)

        # Transpose back to the original structure
        df = df_t.transpose()

        return df
```

plot_one_vs_other function

The function **plot_one_vs_other** is used to create a plot comparing two values, such as GDP to PSGR as shown in earlier examples. The key aspect of this function is to display the data in a specific manner where the **reference country** has two bars of equal height: one representing GDP and the other representing PSGR. This visual representation corresponds to an align_factor of 1. Additionally, the maximum y-limits for both the left and right scales need to be adjusted to ensure that the highest values of colA (e.g., GDP) and colB (e.g., PSGR) are within the plot area.

When modifying the y-limit of the left scale, the y-limit of the right scale should be adjusted by the same factor. This ensures consistent scaling between the two values.

The reference country bars are represented by semi-transparent blue and red colors. When these colors are combined, they result in a magenta color for the bars. If any other colors are visible, it indicates an error in the scale factors used, specifically the reference ratio.

```
In [45]: def plot_one_vs_other(df, colA, colB, ref_country, ref_LEFT, ref_RIGHT, colorA="darkslateblue", colorB="orange", title=None):
        """
        df - dataframe that need to contain index "Country Name"
        colA,colB - column names which will be plotted next to each other (e.g GDP_2018, PSGR_2018)
        ref_country - ref_countryreference country name (e.g "Canada")
        """

        # Create a list of colors for GDP plot
        colA_colors = ['red' if index == ref_country else colorA for index in df.index]
        colB_colors = ['blue' if index == ref_country else colorB for index in df.index]

        # Create a bar plot for GDP
        fig, ax1 = plt.subplots(figsize=(14, 8))
        df[colA].plot(kind='bar', color=colA_colors, ax=ax1, label=colA)

        # Create a new y-axis for PSGR on the right side
        ax2 = ax1.twinx()
        df[colB].plot(kind='bar', color=colB_colors, ax=ax2, alpha=0.5, label=colB)

        # Set title and labels
        if not title:
            title = f'{colA} vs {colB} in {year}'
        plt.title(title)
        ax1.set_ylabel(f'{colA} in [units]')
        ax2.set_ylabel(f'{colB} in [units]')

        # Setting the upper limit of left and right plots
        colA_factor = df[colA].max()/ref_LEFT
        colB_factor = df[colB].max()/ref_RIGHT

        # Factor used depends if factor for colA or colB is larger - the larger factor will be then used on both, left and right scale
        if colA_factor < colB_factor:
            factor = colB_factor
        else:
            factor = colA_factor

        ax1.set_ylim(0, ref_LEFT*factor*1.1)
        ax2.set_ylim(0, ref_RIGHT*factor*1.1)
```

```

# Change color of ref_country label to red
for label in ax1.get_xticklabels():
    if label.get_text() == ref_country:
        label.set_color('red')

# Add Legends
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')

```

```

In [46]: def str_to_percentage(key):
         return str(int(round(float(key) * 100)))+ '%'

```

```

In [47]: def keep_selected_columns(df, *cols_to_keep): #keep_selected_columns
         # This function removes columns that are not specified in the arguments list
         new_df = df[[col for col in df.columns if col in cols_to_keep]]
         new_df = new_df.reindex(columns=cols_to_keep)
         return new_df

```

```

In [48]: def get_stats(df,col_name, world_pop, pop_col_name):
         """Extracts some statistical values from the selected column and returns data combined in the Series"""
         series = df[col_name]
         s_min = pd.Series([series.min()], index=["Min."])
         s_max = pd.Series([series.max()], index=['Max.'])
         s_std = pd.Series([series.std()], index=['Std.'])
         s_count = pd.Series([series.count()], index=["Count"])
         s_mean = pd.Series([series.mean()],index=['Mean'])

         ratio = print_world_population_factor(world_pop, df[pop_col_name],prnt =False)
         s_w_pop_ratio = pd.Series([ratio], index=["world_pop_ratio"])

         s_stats = pd.concat([s_min, s_max, s_std, s_count, s_mean, s_w_pop_ratio])

         return s_stats

```

```

In [49]: def df_row_float_to_int(df, index_name):
         # Transpose the DataFrame
         df_t = df.transpose()

         # Convert the 'world_pop_fract' column to percentage, round to 0 decimal digits, and convert to string
         df_t[index_name] = df_t[index_name].apply(lambda x: int(x) if isinstance(x, float) else x)

```

```
# Transpose back to the original structure
df = df_t.transpose()

return df
```

```
In [50]: def apply_to_all_rows_except(df, excluded_row):
        """
        Converts values in every row to the percentage except excluded row.

        df - input DataFrame.
        excluded_row (str): The name of the row to exclude.

        returns converted dataframe

        """
        for row in df.index:
            if row != excluded_row:
                df = df_row__float_to_percent(df, row)

        return df
```

Extracting data for year 2018

A very important part of the data I collected is the analysis of the align_factor. The key element is to determine how many countries have a factor that is near to the reference country. The selection of the reference country cannot be accidental. Usually, I tend to choose the country that is in the middle of the dataset in terms of the ratio of colA and colB factors.

Below are the threshold factors beyond which the data is discarded. There are three values in my analysis:

- 2.5 - which is equivalent of acceptable deviation of th **250%** of the original reference_ratio,
- 1.0 - which is equivalent of **100%** acceptable deviation
- 0.25 - which is equivalent of **25%** acceptable deviation

```
In [51]: ithr1=2.5 # 250%
        ithr2=1 # 100%
        ithr3=0.25 # 25%
```



```
In [52]: year = 2018

# Getting series for y2018
POP_2018 = get_series_for_selected_year(worldbank__POP, year)
GDP_2018 = get_series_for_selected_year(worldbank__GDP, year)
GDP_PCAP_2018 = get_series_for_selected_year(worldbank__GDP_PCAP, year)
PSGR_2018 = get_series_for_selected_year(worldbank__PSGR, year)
SURF_2018 = get_series_for_selected_year(worldbank__SURF, year)

# Getting worlds population
world_row = worldbank__POP_raw[(worldbank__POP_raw['Country Name'] == 'World')]
world_pop_2018 = int(world_row[str(year)].values[0])
print(f'World population in year {year}: {world_pop_2018}')

# CONCATENATING series into main dataframe
data_2018 = pd.concat([POP_2018, SURF_2018, GDP_PCAP_2018, GDP_2018, PSGR_2018],
                      axis=1,
                      keys=['POP_2018', 'SURF_2018', 'GDP_PCAP_2018', 'GDP_2018', 'PSGR_2018'])

# Droppin NaNs
data_2018.dropna(inplace=True)
```

World population in year 2018: 7661776612

Comparing SURF to PSGR (the country surface area to the air passenger number)

The process of data analysis remains consistent for all subsequent pairs of data:

1. Firstly, the main DataFrame for the year 2018 is copied, and any irrelevant data is removed.
2. Next, the reference country is chosen.
3. The align_factor is then calculated for each country.
4. The values are sorted based on the first parameter.
5. Data that falls outside the acceptable threshold is discarded, and three thresholds are always utilized.
6. A table is generated, presenting the results for different thresholds.
7. Lastly, the plot is displayed.

```

In [53]: SURF_PSGR_2018 = data_2018.copy()
        ## (1)Selected dataframe
        SURF_PSGR_2018 = keep_selected_columns(SURF_PSGR_2018, 'SURF_2018', 'PSGR_2018', 'POP_2018' )

        # (2)Reference country
        ref_country = "China"

        # (3)Generating GDP to SURF align factor
        SURF_PSGR_2018, ref_ratio_SURFvsPSGR, ref_SURF, ref_PSGR =generate_align_factor(SURF_PSGR_2018, "SURF_2018", "PSGR_2018",ref_cou

        # (4)Sorting data by SURF
        SURF_PSGR_2018 = SURF_PSGR_2018.sort_values(by="SURF_2018", ascending=False)

        SURF_PSGR_2018.head(5)

```

```

Out[53]:          SURF_2018  PSGR_2018  POP_2018  SURF_to_PSGR_2018

```

Country Name				
Russian Federation	17098250	99327311.0	144477859	-9.96
Canada	9879750	91504000.0	37065084	-5.88
United States	9831510	889024000.0	326838199	0.42
China	9600012	611439830.0	1402760000	0.00
Brazil	8515770	102109977.0	210166592	-4.31

```

In [54]: # (5)Filtering countries by the GDP to SURF align factor
        _, df_filtered_surf1, _ = exclude_to_high_factor_countries(SURF_PSGR_2018,'SURF_to_PSGR_2018',ithr1)
        _, df_filtered_surf2, _ = exclude_to_high_factor_countries(SURF_PSGR_2018,'SURF_to_PSGR_2018',ithr2)
        _, df_filtered_surf3, _ = exclude_to_high_factor_countries(SURF_PSGR_2018,'SURF_to_PSGR_2018',ithr3)
        # df_f => df_filtered

        args_stats__SURF_vs_PSGR = "SURF_to_PSGR_2018", world_pop_2018, "POP_2018"
        statsAll = get_stats(SURF_PSGR_2018,*args_stats__SURF_vs_PSGR)
        stats1 = get_stats(df_filtered_surf1,*args_stats__SURF_vs_PSGR)
        stats2 = get_stats(df_filtered_surf2,*args_stats__SURF_vs_PSGR)
        stats3 = get_stats(df_filtered_surf3,*args_stats__SURF_vs_PSGR)

        # (6)Creating stats dataframe

```

```

df_input = [statsAll, stats1, stats2, stats3]

index = ["All",
         str_to_percentage(str(ithr1)),
         str_to_percentage(str(ithr2)),
         str_to_percentage(str(ithr3))]

stats_df__SURF_vs_PSGR = pd.DataFrame(df_input, index=index).transpose()

# Converting dataframe rows to different types

# Apply the function to the DataFrame
stats_df__SURF_vs_PSGR = apply_to_all_rows_except(stats_df__SURF_vs_PSGR, "Count")
stats_df__SURF_vs_PSGR = df_row__float_to_int(stats_df__SURF_vs_PSGR, "Count")

# Clearing data
del statsAll, stats1, stats2, stats3, df_input, index

stats_df__SURF_vs_PSGR

```

Out[54]:

	All	250%	100%	25%
Min.	-288092%	-250%	-94%	-19%
Max.	154832%	236%	90%	19%
Std.	31442%	120%	53%	11%
Count	155	57	28	8
Mean	-3645%	-11%	-8%	-3%
world_pop_ratio	96%	61%	49%	22%

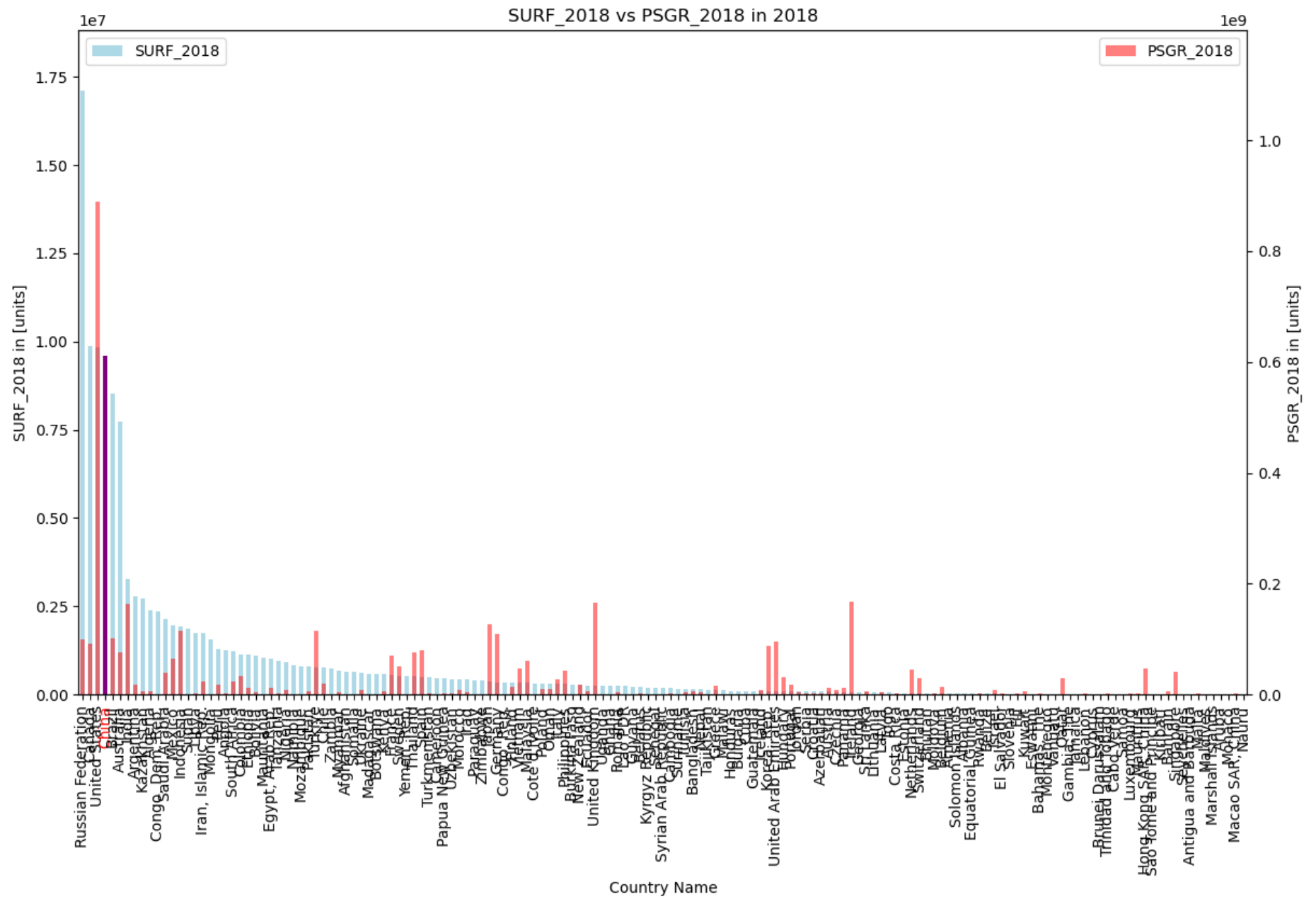
This table provides statistics for each dataframe, excluding data points that fall outside the specified threshold, as well as statistics for the entire dataset.

In the case where a deviation of up to 250% from the reference country (China, in this case) is considered acceptable, approximately 61% of the countries exhibit a similar relationship. When the threshold is reduced to 100%, only 49% of the countries remain (28 of them). Further reducing the threshold to 25% leaves only 22% of the countries (8 countries) in the dataset.

The plot below shows all countries, while the second plot excludes the six largest countries by surface area to emphasize the relationship among smaller countries.

```
In [55]: # Plot for ALL countries
# SURF_PSGR_2018 contains all data
# df_filtered_surf.. contains filtered by threshold data of SURF_PSGR_2018

# (7)Plotting charts
plot_one_vs_other(SURF_PSGR_2018,"SURF_2018", "PSGR_2018", ref_country,ref_SURF, ref_PSGR, colorA="lightblue", colorB="red")
# Plot for ALL countries but 6 largest in SURF
plot_one_vs_other(SURF_PSGR_2018[6:], "SURF_2018", "PSGR_2018", ref_country, ref_SURF, ref_PSGR, colorA="lightblue", colorB="red")
```



only a slight tendency to remain on the left side of the plot. The blue bars represent the surface areas of the countries, and those are sorted.

However, when looking at the plot including the six largest countries (top one), those countries tend to have higher passenger traffic. This observation may explain why, even when using a 100% margin, almost 50% of the population remains within the limits.

Comparing POP to PSGR (population to passenger traffic by air)

The pattern to analyze the data is exactly the same as in the previous case of SURF to PSGR.

```
In [56]: ### Comparing POP to PSGR
POP_PSGR_2018 = data_2018.copy()
## Selected dataframe
POP_PSGR_2018 = keep_selected_columns(POP_PSGR_2018, 'POP_2018', 'PSGR_2018')

# Reference country
ref_country = "Mexico"

# Generating GDP to POP align factor
POP_PSGR_2018, ref_ratio_POPvsPSGR, ref_POP, ref_PSGR = generate_align_factor(POP_PSGR_2018, "POP_2018", "PSGR_2018", ref_country)

# Sorting data by POP
POP_PSGR_2018 = POP_PSGR_2018.sort_values(by="POP_2018", ascending=False)

# POP_PSGR_2018
```

```
In [57]: # Filtering countries by the GDP to POP align factor
_, df_filtered_POP1, _ = exclude_to_high_factor_countries(POP_PSGR_2018, 'POP_to_PSGR_2018', ithr1)
_, df_filtered_POP2, _ = exclude_to_high_factor_countries(POP_PSGR_2018, 'POP_to_PSGR_2018', ithr2)
_, df_filtered_POP3, _ = exclude_to_high_factor_countries(POP_PSGR_2018, 'POP_to_PSGR_2018', ithr3)
# df_f => df_filtered

args_stats__POP_vs_PSGR = "POP_to_PSGR_2018", world_pop_2018, "POP_2018"
statsAll = get_stats(POP_PSGR_2018, *args_stats__POP_vs_PSGR)
stats1 = get_stats(df_filtered_POP1, *args_stats__POP_vs_PSGR)
stats2 = get_stats(df_filtered_POP2, *args_stats__POP_vs_PSGR)
stats3 = get_stats(df_filtered_POP3, *args_stats__POP_vs_PSGR)

# Creating stats dataframe
df_input = [statsAll, stats1, stats2, stats3]
```

```

index = ["All",
         str_to_percentage(str(ithr1)),
         str_to_percentage(str(ithr2)),
         str_to_percentage(str(ithr3))]

stats_df__POP_vs_PSGR = pd.DataFrame(df_input, index=index).transpose()

# Converting dataframe rows to different types

# Apply the function to the DataFrame
stats_df__POP_vs_PSGR = apply_to_all_rows_except(stats_df__POP_vs_PSGR, "Count")
stats_df__POP_vs_PSGR = df_row__float_to_int(stats_df__POP_vs_PSGR, "Count")

# Clearing data
del statsAll, stats1, stats2, stats3, df_input, index

stats_df__POP_vs_PSGR

```

Out[57]:

	All	250%	100%	25%
Min.	-101803%	-239%	-94%	-21%
Max.	6513%	249%	100%	25%
Std.	12702%	117%	49%	12%
Count	155	71	42	16
Mean	-2467%	2%	-12%	-4%
world_pop_ratio	96%	48%	39%	29%

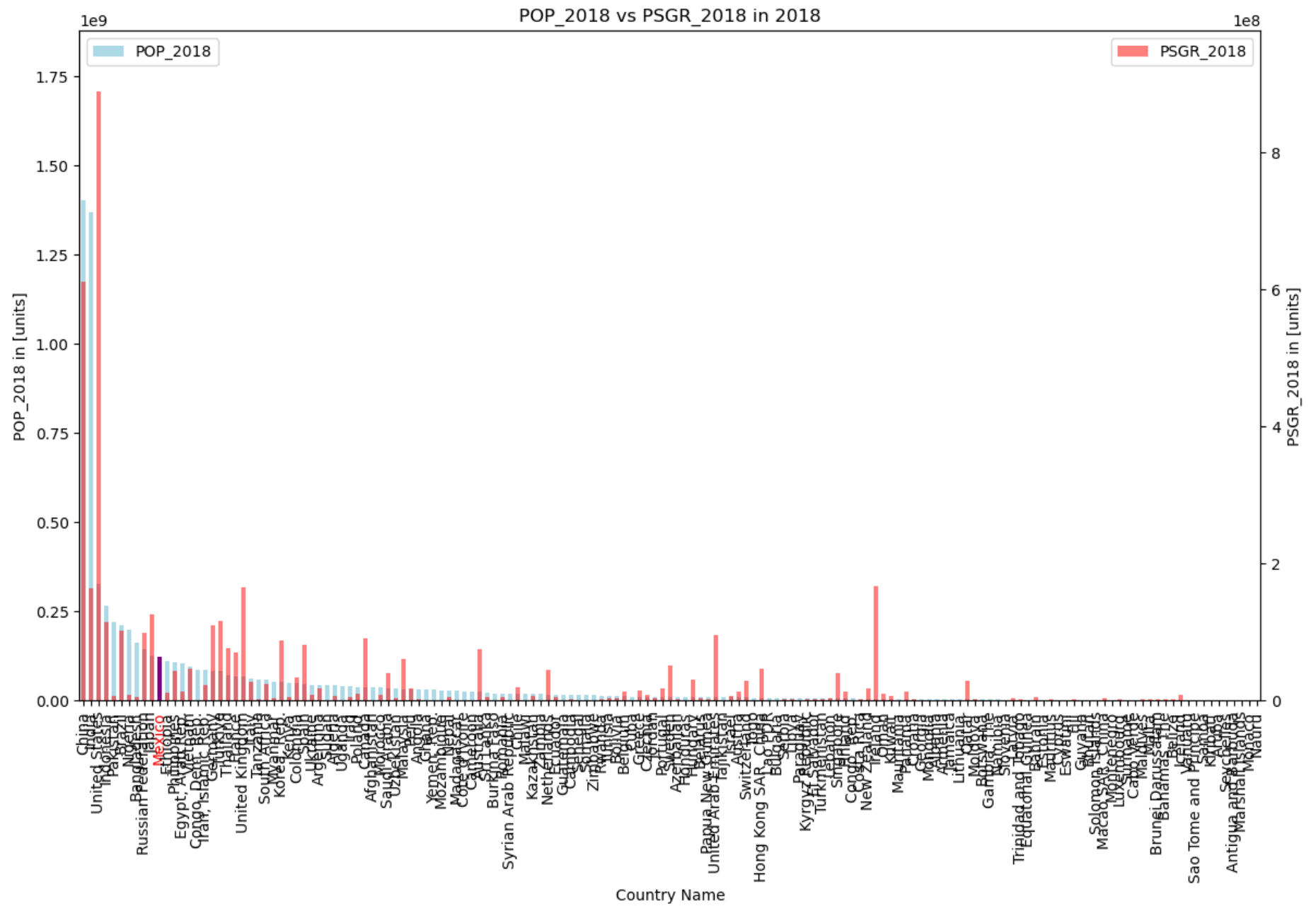
Here, the situation is similar to SURF to PSGR. For the threshold of 250%, 71 countries containing 48% of the population meet the criteria. However, when the limit is reduced to 100%, there are still 42 countries and with 39% of the population within the range. Even with a 25% limit, which is very tight, almost 30% of the population is between the limits.

This may indicate that there is some correlation. When looking at the plots, a high number of passengers traveling tend to be on the left side of the chart.


```
In [58]: # Plot for ALL countries
# POP_PSGR_2018 contains all data
# df_filtered_POP.. contains filtered by threshold data of POP_PSGR_2018

plot_one_vs_other(POP_PSGR_2018,"POP_2018", "PSGR_2018", ref_country,ref_POP, ref_PSGR, colorA="lightblue", colorB="red")
# Plot for ALL countries but 6 largest in POP
plot_one_vs_other(POP_PSGR_2018[6:], "POP_2018", "PSGR_2018", ref_country, ref_POP, ref_PSGR, colorA="lightblue", colorB="red")

# ## Plots for lower thresholds
# plot_one_vs_other(df_filtered_POP1,"POP_2018", "PSGR_2018", ref_country, ref_POP, ref_PSGR)
# plot_one_vs_other(df_filtered_POP2,"POP_2018", "PSGR_2018", ref_country, ref_POP, ref_PSGR)
# plot_one_vs_other(df_filtered_POP3,"POP_2018", "PSGR_2018", ref_country, ref_POP, ref_PSGR)
```



Here again the process is almost identical to previous analysi. Only difference is that population density was calculated using POP and SURF data.

```
In [59]: ### Comparing POP to PSGR
DENS_PSGR_2018 = data_2018.copy()
## Selected dataframe
DENS_PSGR_2018 = keep_selected_columns(DENS_PSGR_2018, 'POP_2018', 'SURF_2018', 'PSGR_2018')

# Calculating population density (DENS)
DENS_PSGR_2018['DENS_2018'] = DENS_PSGR_2018['POP_2018'] / DENS_PSGR_2018['SURF_2018']

# Reference country
ref_country = "Poland"

# Genereating population density (DENS) to number of passenger travel by air (PSGR) align factor
DENS_PSGR_2018, ref_ratio_DENSvsPSGR, ref_DENS, ref_PSGR = generate_align_factor(DENS_PSGR_2018, "DENS_2018", "PSGR_2018", ref_cou

# Sorting data by POP
DENS_PSGR_2018 = DENS_PSGR_2018.sort_values(by="DENS_2018", ascending=False)

max_rows_OFF()
# DENS_PSGR_2018
```

```
In [60]: # Filtering countries by the GDP to POP align factor
args = [DENS_PSGR_2018, 'DENS_to_PSGR_2018']
_, df_filtered_DENS1, _ = exclude_to_high_factor_countries(*args, ithr1)
_, df_filtered_DENS2, _ = exclude_to_high_factor_countries(*args, ithr2)
_, df_filtered_DENS3, _ = exclude_to_high_factor_countries(*args, ithr3)
# df_f => df_filtered

args_stats__DENS_vs_PSGR = "DENS_to_PSGR_2018", world_pop_2018, "POP_2018"
statsAll = get_stats(DENS_PSGR_2018, *args_stats__DENS_vs_PSGR)
stats1 = get_stats(df_filtered_DENS1, *args_stats__DENS_vs_PSGR)
stats2 = get_stats(df_filtered_DENS2, *args_stats__DENS_vs_PSGR)
stats3 = get_stats(df_filtered_DENS3, *args_stats__DENS_vs_PSGR)

# Creating stats dataframe
df_input = [statsAll, stats1, stats2, stats3]

index = ["All",
         str_to_percentage(str(ithr1)),
         str_to_percentage(str(ithr2)),
         str_to_percentage(str(ithr3))]
```

```
stats_df__DENS_vs_PSGR =pd.DataFrame(df_input, index=index).transpose()

# Converting dataframe rows to different types

# Apply the function to the DataFrame
stats_df__DENS_vs_PSGR = apply_to_all_rows_except(stats_df__DENS_vs_PSGR, "Count")
stats_df__DENS_vs_PSGR = df_row_float_to_int(stats_df__DENS_vs_PSGR,"Count")

# Clearing data
del statsAll, stats1, stats2, stats3, df_input, index

stats_df__DENS_vs_PSGR
```

Out[60]:

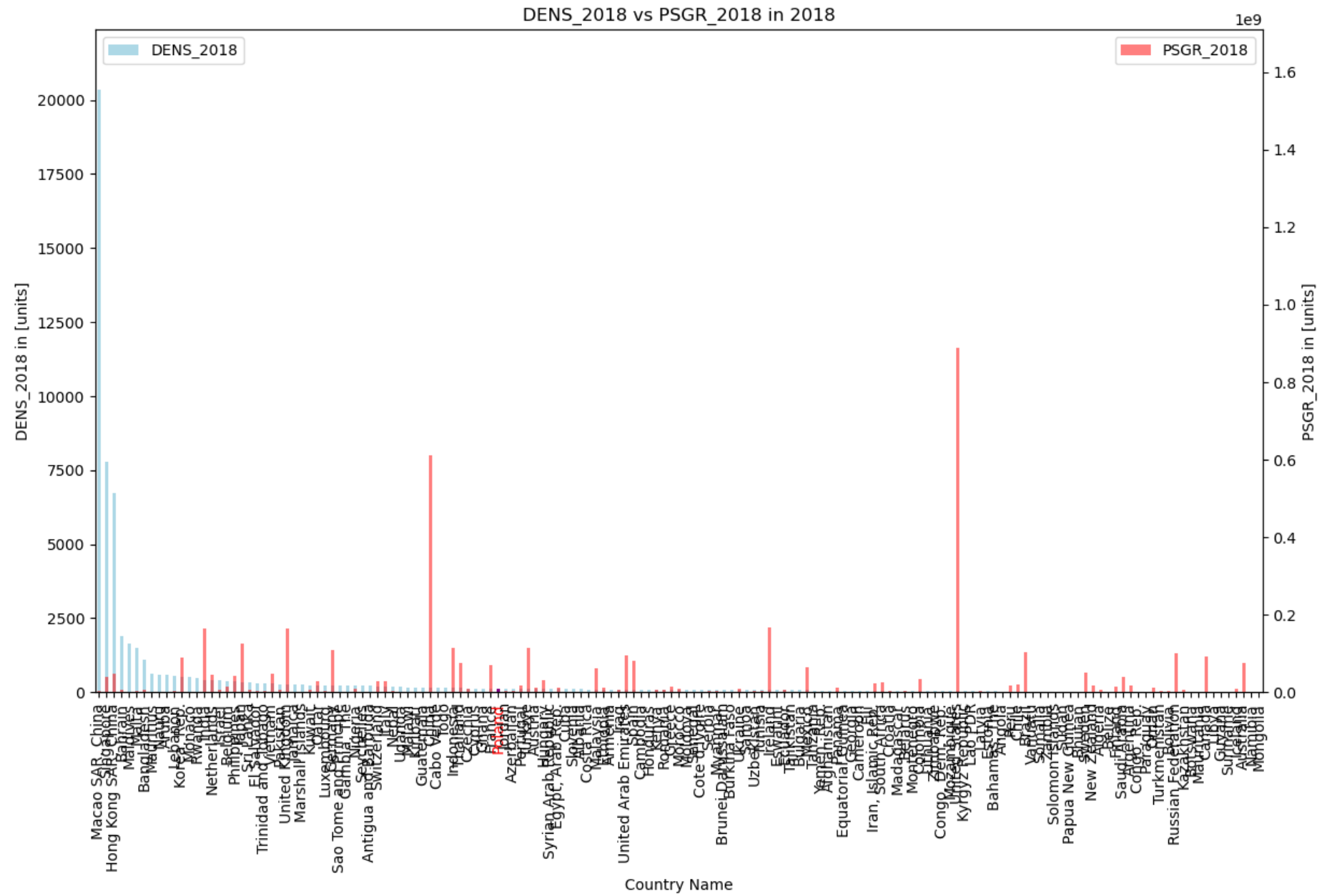
	All	250%	100%	25%
Min.	-12096825%	-227%	-83%	-21%
Max.	34907%	199%	83%	20%
Std.	971505%	106%	48%	14%
Count	155	50	30	9
Mean	-81586%	-17%	5%	-0.3%
world_pop_ratio	96%	21%	11%	1%

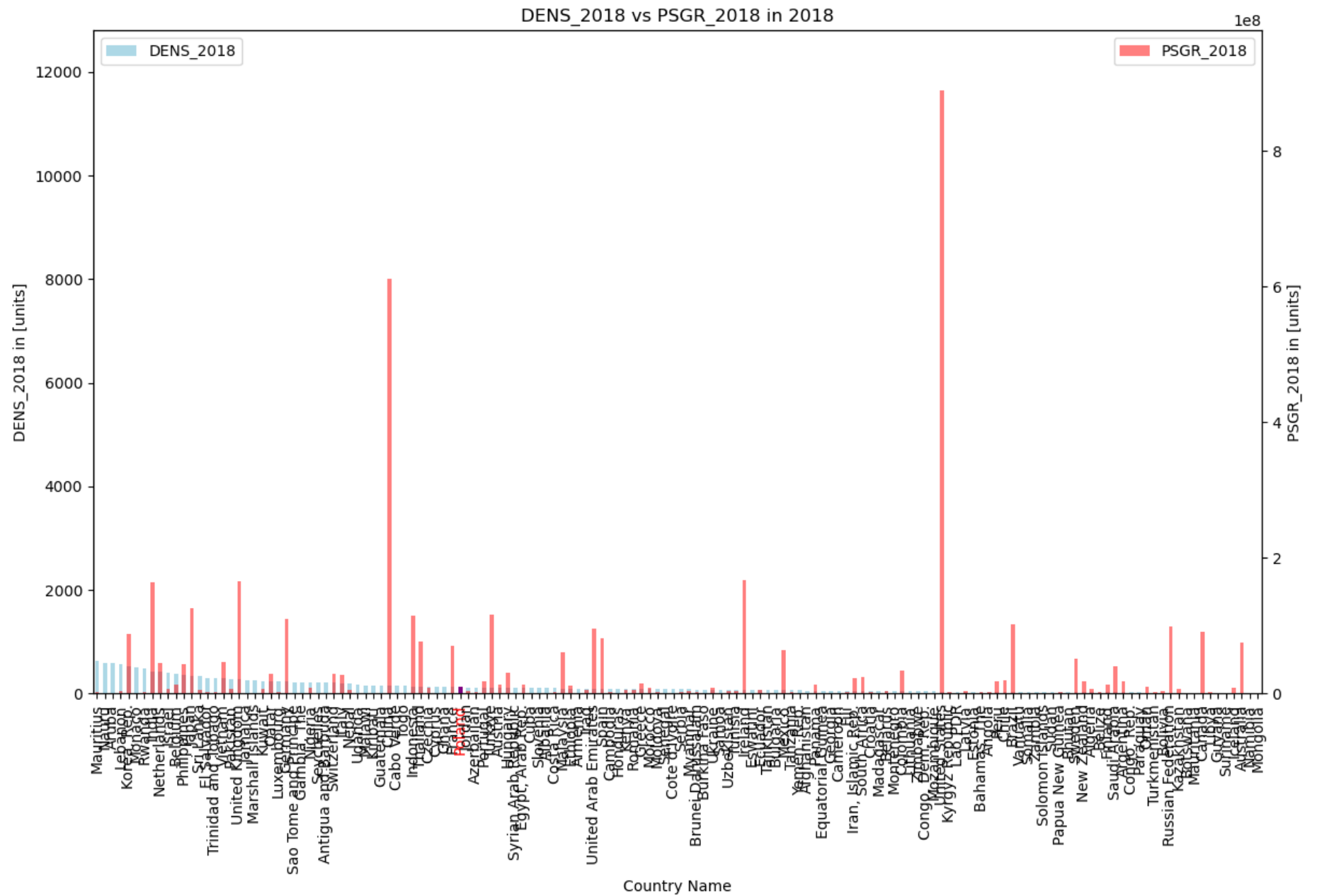
Here, it seems that there is no relation between population density and passenger traffic. Even with a large threshold of 250%, only 21% of the population falls within the limit. For a 25% threshold, there is only 1%.

The distribution of long red bars against the blue ones appears to be random across the entire plot. For eleven largest countries the relation seems to be inverse.

```
In [61]: plot_one_vs_other(DENS_PSGR_2018,"DENS_2018", "PSGR_2018", ref_country,ref_DENS, ref_PSGR, colorA="lightblue", colorB="red")
# Plot for ALL countries but 6 Largest in DENS
plot_one_vs_other(DENS_PSGR_2018[7:], "DENS_2018", "PSGR_2018", ref_country, ref_DENS, ref_PSGR, colorA="lightblue", colorB="red")

# ## Plots for lower thresholds
# plot_one_vs_other(df_filtered_DENS1,"DENS_2018", "PSGR_2018", ref_country, ref_DENS, ref_PSGR)
# plot_one_vs_other(df_filtered_DENS2,"DENS_2018", "PSGR_2018", ref_country, ref_DENS, ref_PSGR)
# plot_one_vs_other(df_filtered_DENS3,"DENS_2018", "PSGR_2018", ref_country, ref_DENS, ref_PSGR)
```





Comparing GDP_PCAP to PSGR (GPD per capita to passenger traffic)

Same analysis proces as before.

```
In [62]: GDP_PCAP_PSGR_2018 = data_2018.copy()
        ## Selected dataframe
        GDP_PCAP_PSGR_2018 = keep_selected_columns(GDP_PCAP_PSGR_2018, 'GDP_PCAP_2018', 'PSGR_2018', 'POP_2018' )

        # Reference country
        ref_country = "Spain"

        # Genereating GDP to GDP_PCAP align factor
        GDP_PCAP_PSGR_2018, ref_ratio_GDP_PCAPvsPSGR, ref_GDP_PCAP, ref_PSGR =generate_align_factor(GDP_PCAP_PSGR_2018, "GDP_PCAP_2018",

        # Sorting data by GDP_PCAP
        GDP_PCAP_PSGR_2018 = GDP_PCAP_PSGR_2018.sort_values(by="GDP_PCAP_2018", ascending=False)

        # GDP_PCAP_PSGR_2018
```

```
In [63]: # Filtering countries by the GDP to GDP_PCAP align factor
        _, df_filtered_GDP_PCAP1, _ = exclude_to_high_factor_countries(GDP_PCAP_PSGR_2018, 'GDP_PCAP_to_PSGR_2018', ithr1)
        _, df_filtered_GDP_PCAP2, _ = exclude_to_high_factor_countries(GDP_PCAP_PSGR_2018, 'GDP_PCAP_to_PSGR_2018', ithr2)
        _, df_filtered_GDP_PCAP3, _ = exclude_to_high_factor_countries(GDP_PCAP_PSGR_2018, 'GDP_PCAP_to_PSGR_2018', ithr3)
        # df_f => df_filtered

        args_stats__GDP_PCAP_vs_PSGR = "GDP_PCAP_to_PSGR_2018", world_pop_2018, "POP_2018"
        statsAll = get_stats(GDP_PCAP_PSGR_2018,*args_stats__GDP_PCAP_vs_PSGR)
        stats1 = get_stats(df_filtered_GDP_PCAP1,*args_stats__GDP_PCAP_vs_PSGR)
        stats2 = get_stats(df_filtered_GDP_PCAP2,*args_stats__GDP_PCAP_vs_PSGR)
        stats3 = get_stats(df_filtered_GDP_PCAP3,*args_stats__GDP_PCAP_vs_PSGR)

        # Creating stats dataframe
        df_input = [statsAll, stats1, stats2, stats3]

        index = ["All",
                  str_to_percentage(str(ithr1)),
                  str_to_percentage(str(ithr2)),
                  str_to_percentage(str(ithr3))]

        stats_df__GDP_PCAP_vs_PSGR =pd.DataFrame(df_input, index=index).transpose()

        # Converting dataframe rows to different types
```



```
# Apply the function to the DataFrame
stats_df__GDP_PCAP_vs_PSGR = apply_to_all_rows_except(stats_df__GDP_PCAP_vs_PSGR, "Count")
stats_df__GDP_PCAP_vs_PSGR = df_row__float_to_int(stats_df__GDP_PCAP_vs_PSGR, "Count")

# Clearing data
del statsAll, stats1, stats2, stats3, df_input, index

stats_df__GDP_PCAP_vs_PSGR
```

Out[63]:

	All	250%	100%	25%
Min.	-164688817%	-244%	-89%	-18%
Max.	2881%	233%	93%	21%
Std.	13227707%	109%	46%	14%
Count	155	51	29	10
Mean	-1067824%	-64%	-1%	4%
world_pop_ratio	96%	33%	24%	10%

Some slight relationship can be noticed, but it is rather weak. Only 33%, 24%, and 10% of the population respectively meet the criteria for each threshold. When looking at the plot, the distribution is similar to the surface to passenger relationship. When looking on the plot the red bars tend to remain at the right side of the plot.

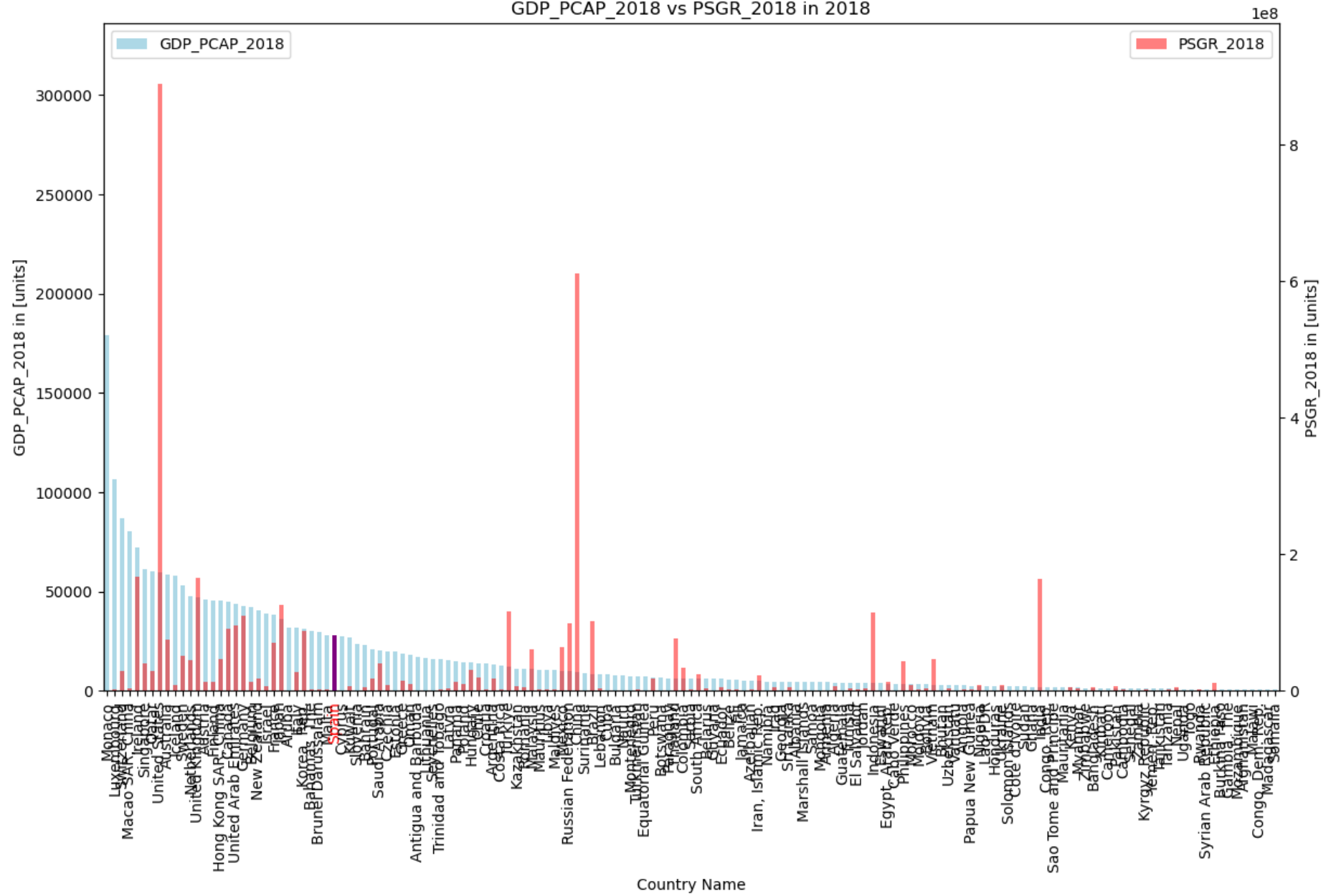
In [64]:

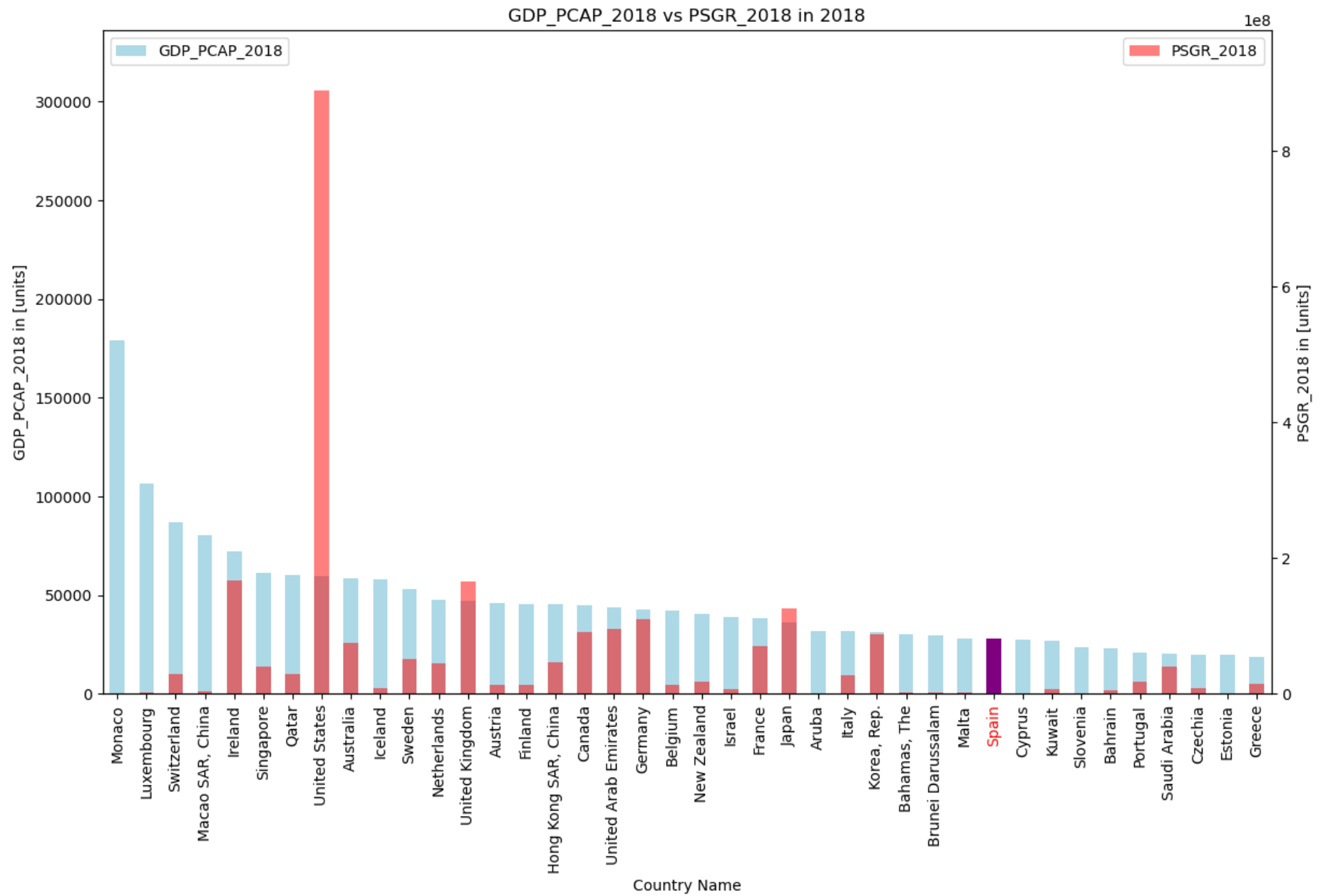
```
# Plot for ALL countries
# GDP_PCAP_PSGR_2018 contains all data
# df_filtered_GDP_PCAP.. contains filtered by threshold data of GDP_PCAP_PSGR_2018

plot_one_vs_other(GDP_PCAP_PSGR_2018, "GDP_PCAP_2018", "PSGR_2018", ref_country, ref_GDP_PCAP, ref_PSGR, colorA="lightblue", colorB="red")
# Plot for ALL countries but 6 largest in GDP_PCAP
plot_one_vs_other(GDP_PCAP_PSGR_2018[:40], "GDP_PCAP_2018", "PSGR_2018", ref_country, ref_GDP_PCAP, ref_PSGR, colorA="lightblue", colorB="red")

# ## Plots for lower thresholds
# plot_one_vs_other(df_filtered_GDP_PCAP1, "GDP_PCAP_2018", "PSGR_2018", ref_country, ref_GDP_PCAP, ref_PSGR)
# plot_one_vs_other(df_filtered_GDP_PCAP2, "GDP_PCAP_2018", "PSGR_2018", ref_country, ref_GDP_PCAP, ref_PSGR)
# plot_one_vs_other(df_filtered_GDP_PCAP3, "GDP_PCAP_2018", "PSGR_2018", ref_country, ref_GDP_PCAP, ref_PSGR)
```

GDP_PCAP_2018 vs PSGR_2018 in 2018





GDP to PSGR, year 2018

The final relation of GDP to the passenger count. Process is still the same as before.

```
In [65]: GDP_PSGR_2018 = data_2018.copy()
        ## Selected dataframe
        GDP_PSGR_2018 = keep_selected_columns(GDP_PSGR_2018, 'GDP_2018', 'PSGR_2018', 'POP_2018' )
```

```
In [66]: # Reference country
        ref_country = "Canada"

        # Generating GDP to SURF align factor
        GDP_PSGR_2018, ref_ratio_GDPvsPSGR, ref_GDP, ref_PSGR = generate_align_factor(GDP_PSGR_2018, "GDP_2018", "PSGR_2018", ref_country)

        # Sorting data by SURF
        GDP_PSGR_2018 = GDP_PSGR_2018.sort_values(by="GDP_2018", ascending=False)

        # GDP_PSGR_2018
```

```
In [67]: # Filtering countries by the GDP to GDP align factor
        _, df_filtered_GDP1, _ = exclude_to_high_factor_countries(GDP_PSGR_2018, 'GDP_to_PSGR_2018', ithr1)
        _, df_filtered_GDP2, _ = exclude_to_high_factor_countries(GDP_PSGR_2018, 'GDP_to_PSGR_2018', ithr2)
        _, df_filtered_GDP3, _ = exclude_to_high_factor_countries(GDP_PSGR_2018, 'GDP_to_PSGR_2018', ithr3)
        # df_f => df_filtered
```

```
In [68]: args_stats__GDP_vs_PSGR = "GDP_to_PSGR_2018", world_pop_2018, "POP_2018"
        statsAll = get_stats(GDP_PSGR_2018, *args_stats__GDP_vs_PSGR)
        stats1 = get_stats(df_filtered_GDP1, *args_stats__GDP_vs_PSGR)
        stats2 = get_stats(df_filtered_GDP2, *args_stats__GDP_vs_PSGR)
        stats3 = get_stats(df_filtered_GDP3, *args_stats__GDP_vs_PSGR)

        # Creating stats dataframe
        df_input = [statsAll, stats1, stats2, stats3]

        index = ["All",
                  str_to_percentage(str(ithr1)),
                  str_to_percentage(str(ithr2)),
                  str_to_percentage(str(ithr3))]

        stats_df__GDP_vs_PSGR = pd.DataFrame(df_input, index=index).transpose()
```

```
# Converting dataframe rows to different types

# Apply the function to the DataFrame
stats_df__GDP_vs_PSGR = apply_to_all_rows_except(stats_df__GDP_vs_PSGR, "Count")
stats_df__GDP_vs_PSGR = df_row__float_to_int(stats_df__GDP_vs_PSGR, "Count")

# Clearing variables
del statsAll, stats1, stats2, stats3, index, df_input
stats_df__GDP_vs_PSGR
```

Out[68]:

	All	250%	100%	25%
Min.	-120647%	-244%	-92%	-22%
Max.	957%	218%	94%	25%
Std.	9776%	110%	46%	13%
Count	155	117	75	39
Mean	-1063%	1%	4%	1%
world_pop_ratio	96%	90%	65%	57%

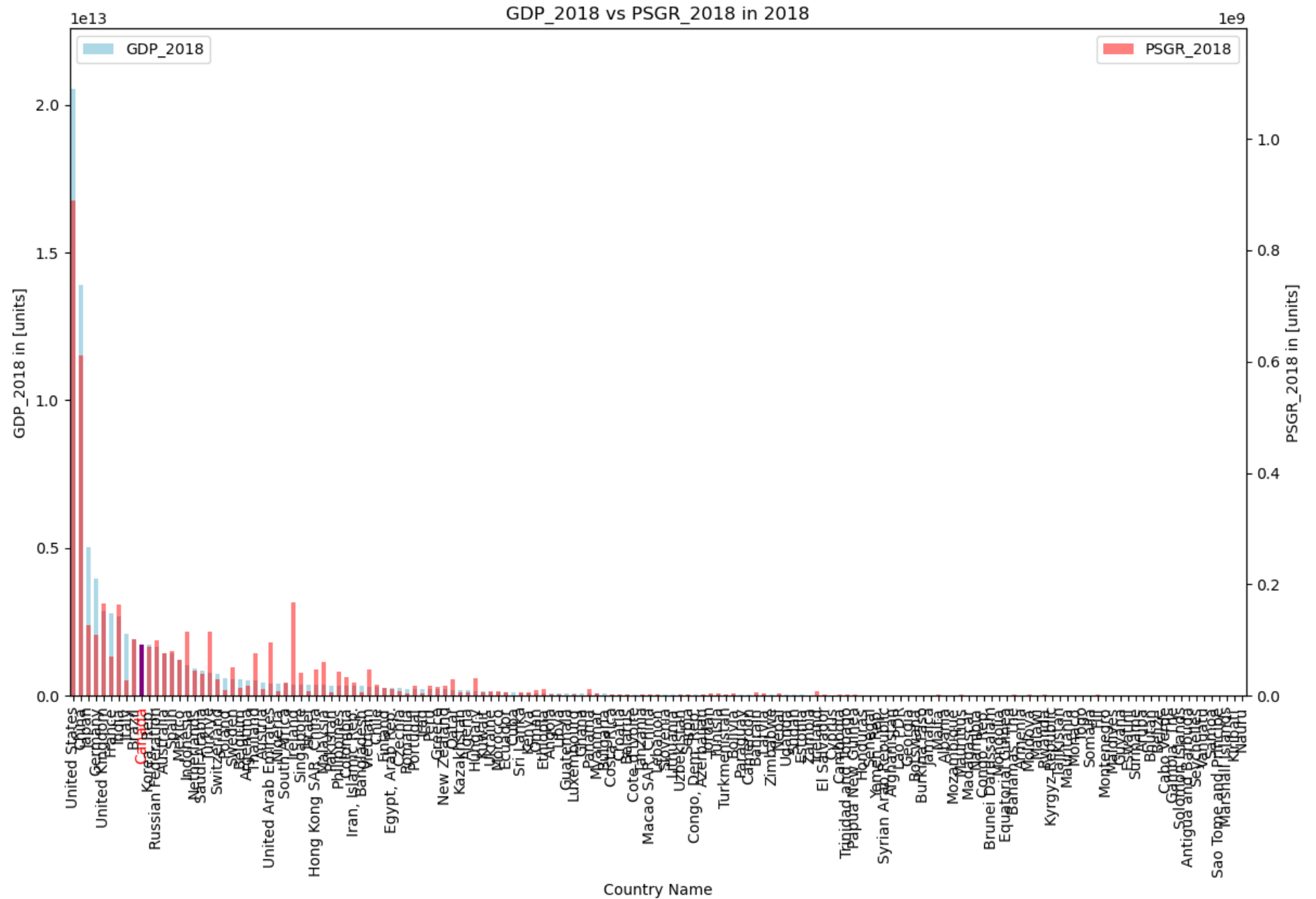
For the GDP to PSGR relationship, there appears to be a strong relationship. Even with a restrictive threshold of 25%, there are still 39 countries within the limit, accounting for a total of 57% of the population. When the threshold is increased to 250%, 117 countries and 90% of the population fall within the range. This correlation is also reflected in the plot below, where the longest red bars closely follow the declining shape of the blue bars.

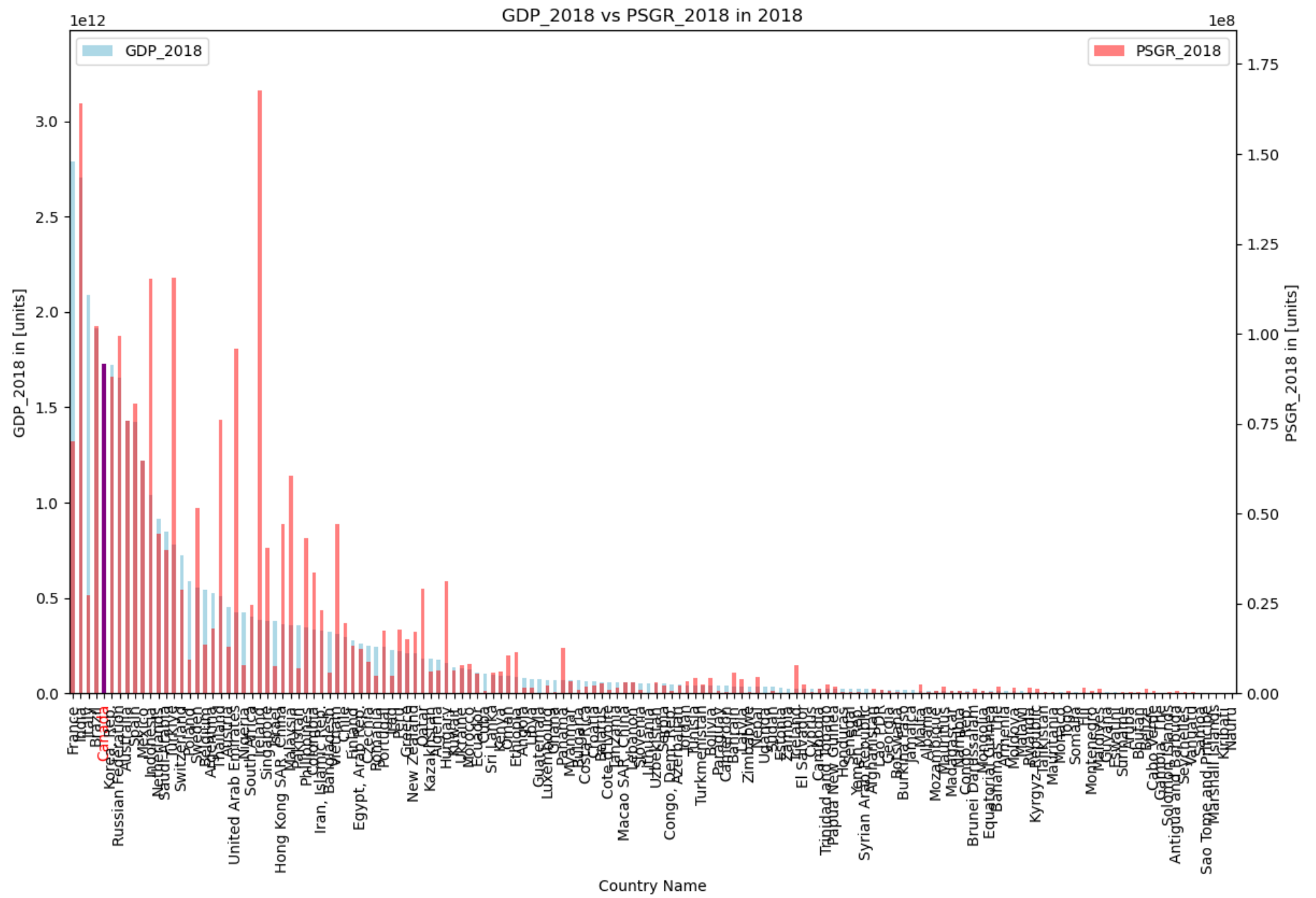
```
In [69]: # Plot for ALL countries
# GDP_PSGR_2018 contains all data
# df_filtered_GDP.. contains filtered by threshold data of GDP_PSGR_2018

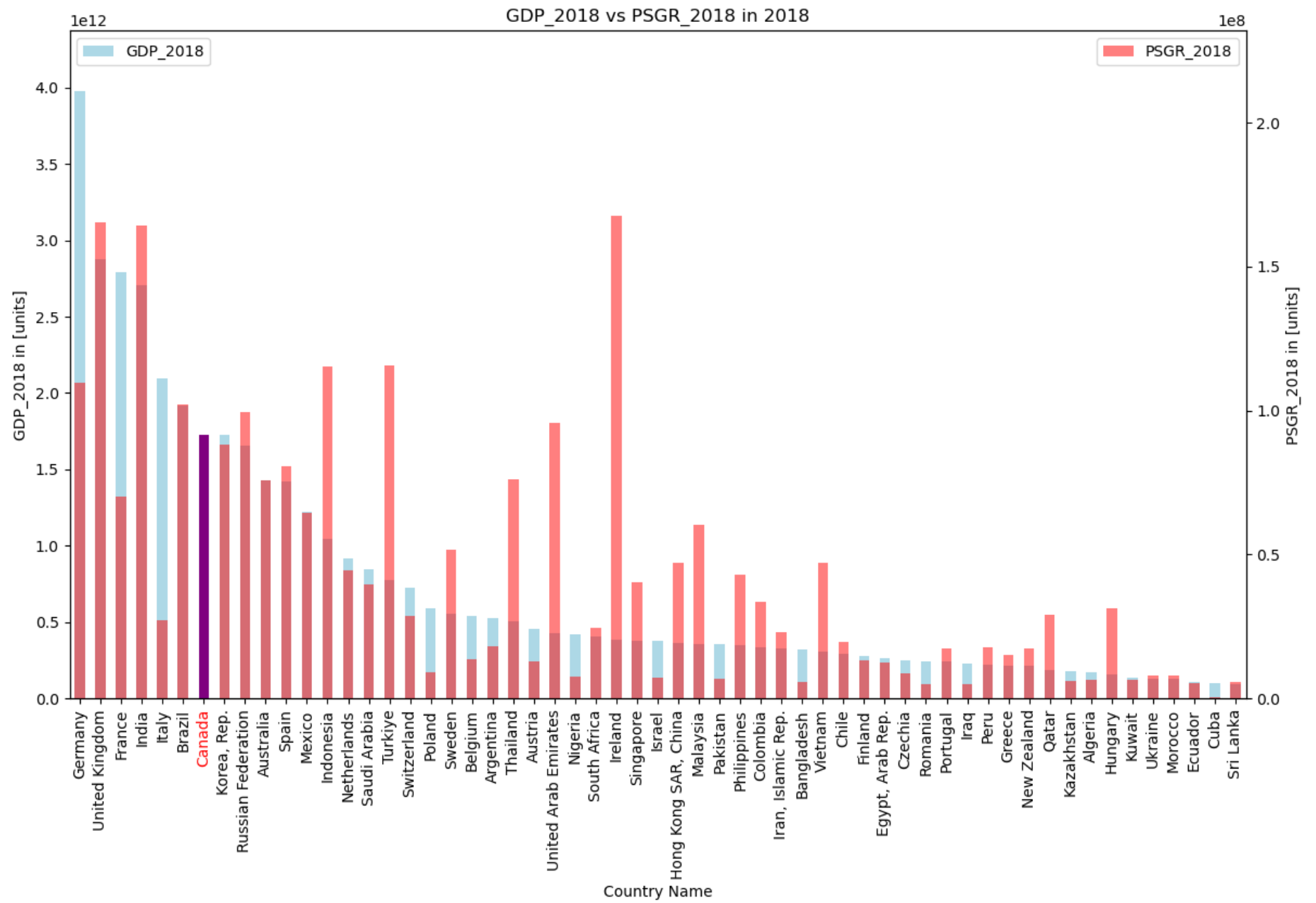
args = ["GDP_2018", "PSGR_2018", ref_country, ref_GDP, ref_PSGR]
kwargs = {"colorA": "lightblue", "colorB": "red"}

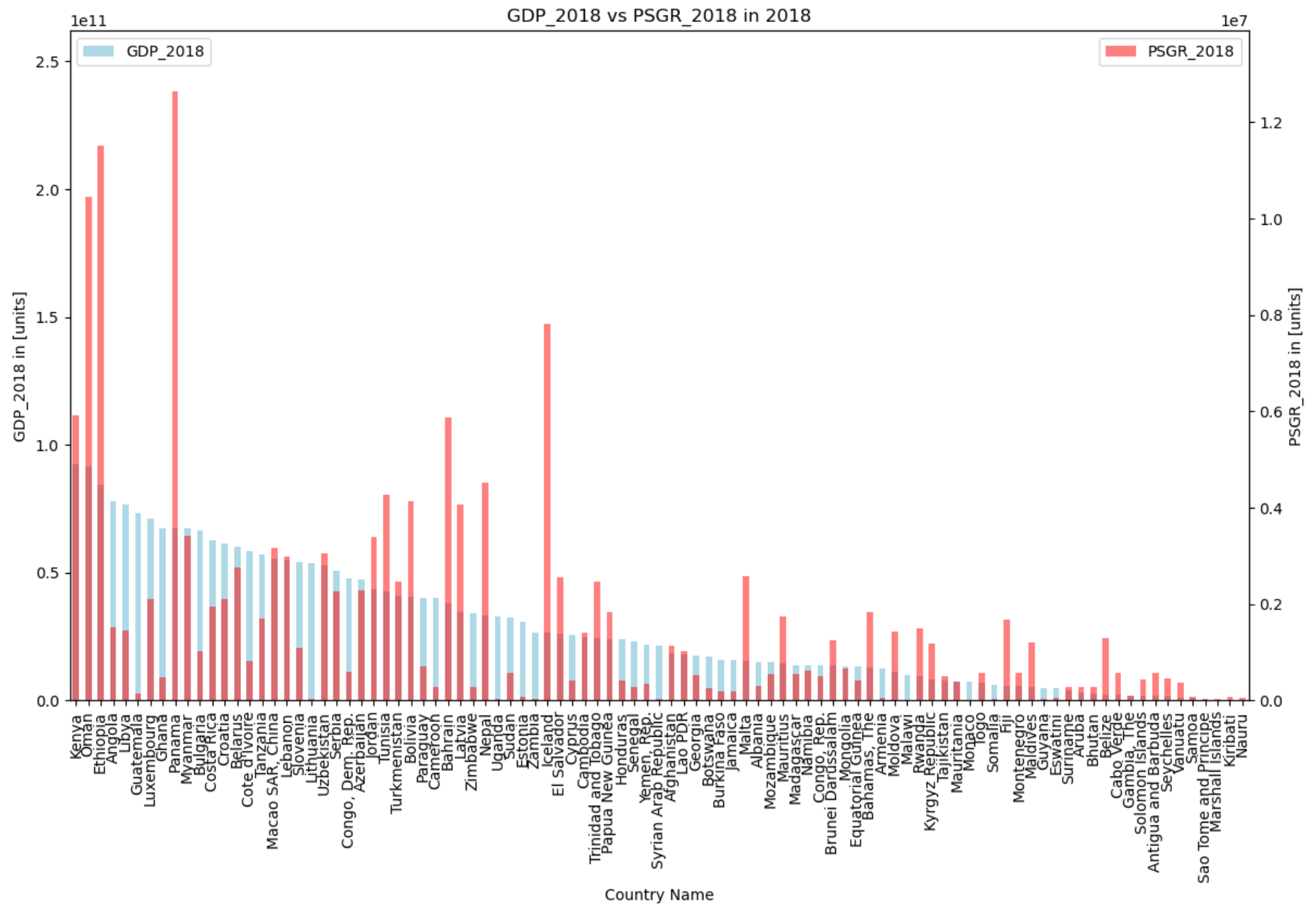
plot_one_vs_other(GDP_PSGR_2018, *args, **kwargs)
plot_one_vs_other(GDP_PSGR_2018[5:], *args, **kwargs)
# Plot for ALL countries but 3 largest in GDP
plot_one_vs_other(GDP_PSGR_2018[3:60], *args, **kwargs)
```

```
plot_one_vs_other(GDP_PSGR_2018[60:], *args, **kwargs)
del args,kwargs
```









The first plot includes all countries.

The second plot includes all countries, except the four with the largest economy.

The third plot excludes the two largest economies and goes up to the 60th one.

The last plot starts at the 60th economy and includes all remaining countries.

Conclusions

Through data analysis, it can be concluded that GDP can be regarded as a highly reliable predictor of passenger traffic. This relationship is supported by some online articles, such as the one found at <https://www.ishkaglobal.com/News/Article/5515/GDP-growth-vs-air-travel-A-conscious-uncoupling>.

Furthermore, it is interesting to note that factors like population and country size have limited impact on passenger traffic, despite the viability of air travel especially in large countries.

On the other hand, the development of air travel relies on the availability of expensive infrastructure, which becomes more accessible as a country's economy grows. Additionally, as societies become wealthier, air travel becomes more affordable and accessible to a larger segment of the population.

The correlation between GDP and passenger traffic is an interesting discovery for me. When I started this project, I did not know what to expect or if I would be able to find any meaningful relationships.

The use of tabular data and plots was crucial in understanding the results. Tables provided hard evidence of some relationships, and data visualization helped to gain a sense of the data that was not available solely by looking at the numbers.

In terms of coding, I found that data manipulation was more challenging and time-consuming than I had anticipated. Additionally, using Jupyter Notebook presented some difficulties. It was harder to navigate compared to using a regular integrated development environment, and there were instances where remaining data in memory could give access to variables that I had discarded.

However, on the positive side, Jupyter Notebook made it easier to access specific parts of the data. I could run only a portion of the code instead of the entire code, which was helpful. Nonetheless, I had to be cautious about any data remaining in memory that could potentially distort the process. Additionally, the ability to easily mix code with text and plots is something that Jupyter Notebook makes easy.

Ideas for further analysis

As mentioned earlier, the airport dataframe was not utilized as initially intended in this project due to time constraints for the midterm submission.

However, for further development, it would be compelling to explore whether there is a correlation between higher GDP and the availability of airports for airline use. Additionally, by analyzing the airport data, it would be interesting to examine the distribution of small and medium airports in relation to large airports. In this analysis, the political regime dataframe could be used to see how the political regime affects the expansion of small aviation, specifically General Aviation, by counting the number of small airports. Below, I provide an example of China versus the US.

Another potential avenue for comparison could involve analyzing airport utilization by considering factors such as the number of runways or the total length of runways. To facilitate this exploration, a separate dataframe specifically for runways has been prepared. I used the runways dataframe to present the results of my initial exploration of that dataframe. The short results are presented below.

Additional code

Merging airports dataframe with runway dataframe

Here I merge the runway and airport data to be able to extract the airports which are used by airlines.

```
In [70]: ### MERGING airports and runways dataframes  
apt_and_rwys= pd.merge(airports, runways, left_on='ident', right_on='airport_ident')  
apt_and_rwys.shape
```

```
Out[70]: (28331, 43)
```

```
In [71]: # Removing "airport_ident" column from the dataframe  
apt_and_rwys2 = apt_and_rwys.drop('airport_ident', axis=1)
```

```
In [72]: # Drop not required columns  
apt_and_rwys3 = apt_and_rwys2.drop(['id_x', 'local_region', 'gps_code', 'home_link', 'keywords', 'score', 'id_y', 'last_updated', 'local_
```

```
In [73]: apt_and_rwys3.shape
```

```
Out[73]: (28331, 32)
```

```
In [74]: # [x for x in apt_and_rwys3.head(0)]
```

Extracting airports used by airlines and storing them in the json file.

```
In [75]: schedule_service_only_apt = apt_and_rwys3[apt_and_rwys3['scheduled_service'] == 1]
stripped = schedule_service_only_apt["ident"].squeeze()
only_scheduled_services_apt_ident = stripped.drop_duplicates()

only_scheduled_services_apt_ident.to_json("Modified_data/only_scheduled.json", orient="records")
```

```
In [76]: import json
with open('Modified_data/only_scheduled.json', "r") as file:
    data = json.load(file)
```

Here, check what classification of the airports can be found in the database. I mentioned a few times small, medium, and large airports. Here is the source of this information.

```
In [77]: idents_only = schedule_service_only_apt.drop_duplicates(subset='ident')
# Checking unique values for some columns
unique_values = set(apt_and_rwys3['type'].unique())
unique_values
```

```
Out[77]: {'balloonport',
'closed',
'heliport',
'large_airport',
'medium_airport',
'seaplane_base',
'small_airport'}
```

```
In [78]: # I considered to group the airports in terms of "municipality "
```

```
In [79]: # # Since there is more than fifteen thousand of NaNs in "continent" column, that makes it unreliable data, so I remove it from the data
# apt_and_rwys4 = apt_and_rwys3.drop([ "municipality"], axis=1)
# # apt_and_rwys4.head(5)
```

Removing redundant data.

```
In [80]: apt_and_rwys5 = pd.DataFrame(apt_and_rwys3, columns=['ident', 'iata_code', 'type', 'iso_country', 'country_name'])
```

Grouping data

```
In [81]: types = apt_and_rwys5.groupby(['country_name', 'type']).size()
```

China vs US number of airports comparision

Below, the comparison of the number of each class of airports between the United States and China can be seen.

```
In [82]: types["United States"]
```

```
Out[82]: type
balloonport      3
closed          1099
heliport         179
large_airport    238
medium_airport   1656
seaplane_base    114
small_airport   10330
dtype: int64
```

```
In [83]: types["China"]
```

```
Out[83]: type
closed           4
large_airport    67
medium_airport   176
small_airport    59
dtype: int64
```

The difference is very significant, especially when considering small airports. Despite both countries being nearly equal in size, the numbers are vastly different. While the GDP of both countries may not differ significantly, the disparity in the number of airports is substantial, with a difference of tens or even hundreds.

In this context, it would be valuable to compare the political regimes of these countries and explore if any patterns emerge in other countries as well.

Runways - example for further exploration

Below, there are two plots:

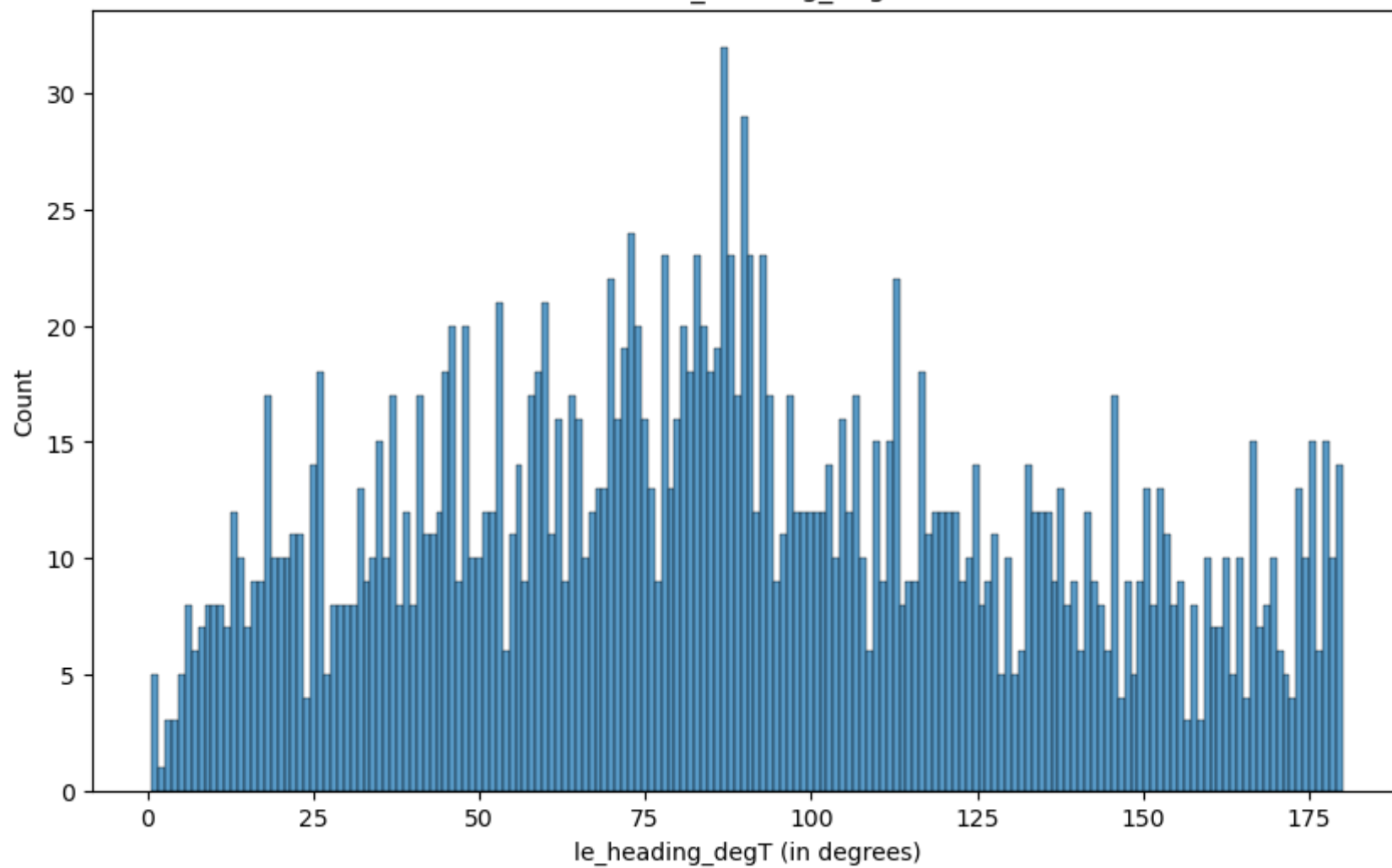
1. The first plot shows the distribution of runway directions worldwide.
2. The second plot focuses specifically on the United States, displaying the distribution of runway directions within the country.

```
In [84]: df = apt_and_rwys3
df = df[df['continent'] == 'EU']
df = df[df['le_heading_degT'].between(0, 180)]

# now create a histogram with seaborn
plt.figure(figsize=(10,6))
sns.histplot(df['le_heading_degT'], bins=180, kde=False)

plt.title('Distribution of le_heading_degT - WORLD')
plt.xlabel('le_heading_degT (in degrees)')
plt.ylabel('Count')
plt.show()
```

Distribution of le_heading_degT - WORLD



From the plot, it can be observed that the bars, representing runway direction, tend to accumulate near the middle and decrease in length as they approach the 0 and 180-degree ends. I chose to use the range of 0 to 180 degrees only, excluding 360 degrees, as including the full circle of data would be redundant. Each runway has two ends, so this plot displays only the "low value" end.

Regarding the accumulation near the center, one possible reason is the prevailing wind direction on the planet. The Earth's rotation influences the global wind patterns, leading to the prevailing wind direction for atmospheric air masses in the mid-latitudes being generally from the west. This is influenced by the Coriolis effect (the apparent deflection of the path of an object that moves within a rotating coordinate system).

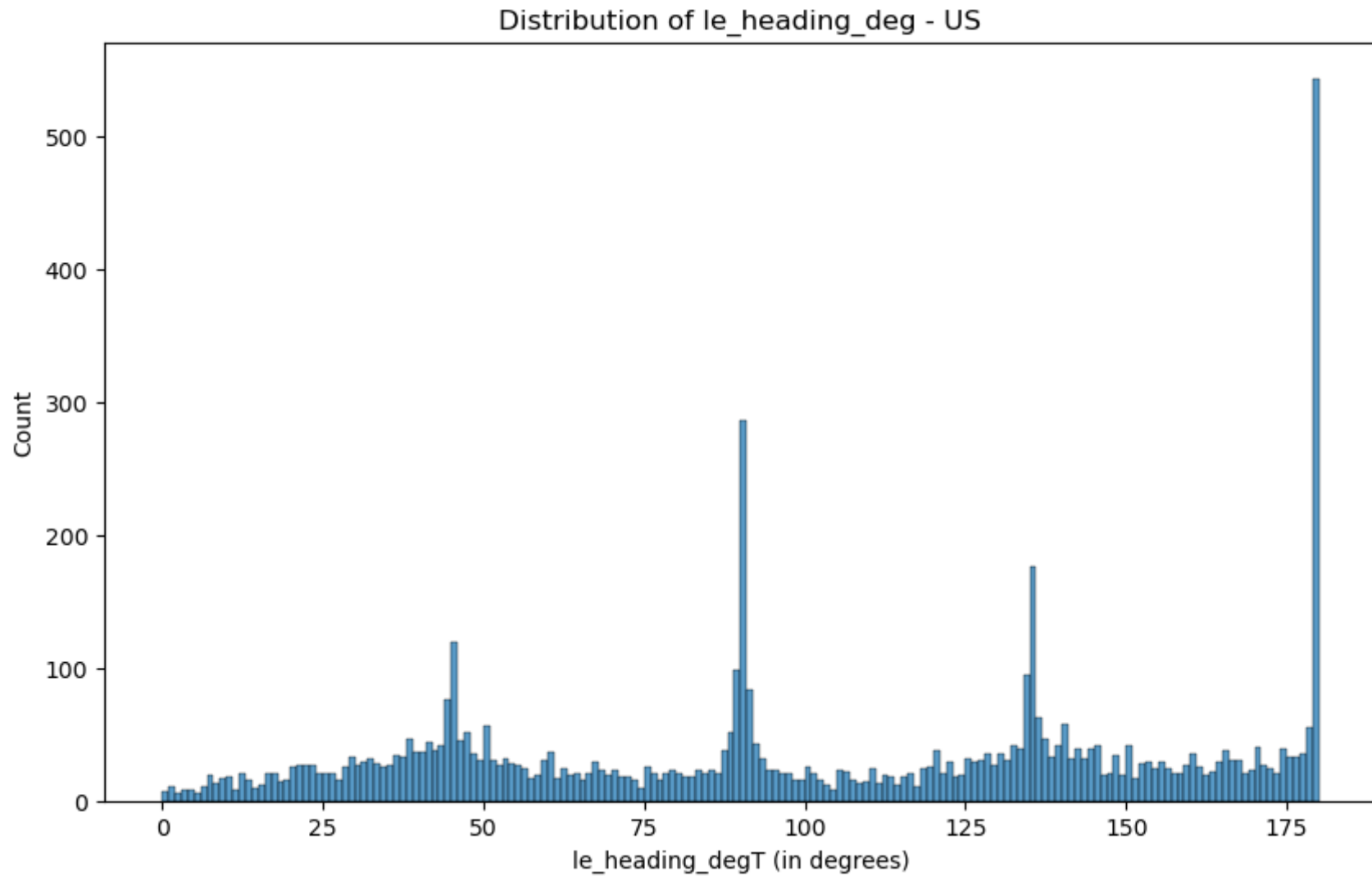
```
In [85]: df = apt_and_rwys3
df = df[df['country_name'] == 'United States']
# df = df[df['le_heading_degT'].between(0, 180)]

## FILTERING AIRPORTS WHICH HAS le heading and no he heading
df = df[df['le_heading_degT'].between(0, 180) & df['he_heading_degT'].notna()]
# df = df[df['he_heading_degT'].notna()]
print(df.shape)

# now create a histogram with seaborn
plt.figure(figsize=(10,6))
sns.histplot(df['le_heading_degT'], bins=180, kde=False)

plt.title('Distribution of le_heading_deg - US')
plt.xlabel('le_heading_degT (in degrees)')
plt.ylabel('Count')
plt.show()
```

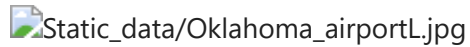
```
(5875, 32)
```

Finally, the plot above shows the runway directions in the United States. It is quite common in the US to use cardinal directions for building streets or establishing state lines. This can be seen as spikes in the 0, 45, 90, 135, and 180-degree directions.

Below, the Google Maps image shows how the runways are positioned. It also shows how the surrounding streets follow the North-South and East-West directions.

 Static_data/Oklahoma_airport_surrounding.jpg



Ethics considerations

Earlier, I provided the sources of my data and the Terms of Use for each dataset. Statistics regarding each country, such as population, surface area, and density, are widely used metrics. As for the data regarding airports and runways, this information is maintained by enthusiasts, not by any institution, so accuracy may not always be guaranteed. In the case of any commercial use of such data, this should be taken into account.

I think that the dataset on political regimes, which I haven't used, could be deemed controversial in some circumstances. Not everyone can agree on the classification of a country as authoritarian or democratic. For the citizens of countries regarded as authoritarian, this might not always be acceptable.

In my datasets, there are no concerns about data privacy or confidentiality. Regarding potential bias in the project, from a data perspective, the markers I used represent a wide range of statistics, so there probably isn't an issue there. I've tried to avoid any personal bias related to the interpretation of my results.